취약점 진단 결과 보고서

모바일 취약점 진단

[SK쉴더스 루키즈 / 31기]

해킹의 민족

진단 기간: 2026.07.25 ~ 2026.08.05

진단 대상: 해킹의 민족(com.hackmin.app) Android App

문서 등급: 대외비

목 차

1. 사업 개요 1
1.1. 사업 배경 및 목적 1
1.2. 진단 대상 및 범위 1
1.3. 수행 일정 1
1.4. 수행 인원 및 역할 2
2. 진단 수행 방법 3
2.1. 사전 협의 3
2.2. 계획 수립 3
2.3. 진단 절차 3
2.4. 진단 도구 4
2.5. 진단 기준 근거 및 출처 4
2.6. 진단 항목 5
3. 진단 결과 요약 7
3.1. 총평 7
3.2. 진단 결과 통계 7
3.3. 결과 분석 및 시사점 7
3.4. 진단 항목별 결과 목록 7
4. 상세 진단 결과 9
4.1. OS 변조(루팅/탈옥) 탐지 9
4.2. 디버깅 탐지 및 차단 12
4.3. 소스코드 난독화 13
4.4. 동적 분석 도구 탐지 19
4.5. 보안 프로그램 및 에뮬레이터 탐지 22
4.6. 단말기 내 중요정보 로컬 저장 방지 24
4.7. 앱 소스코드 내 운영정보 하드코딩 26
4.8. 메모리 내 중요정보 노출 방지 27
4.9. 디버그 로그 내 중요정보 노출 29

4.10. 화면 캡처 방지 및 중요정보 마스킹 31
4.11. 백그라운드 화면 전환 시 정보 보호 34
4.12. 앱 백업(AllowBackup) 설정 보안 36
4.13. 외부 저장소(External Storage) 접근 통제 38
4.14. 통신 구간 암호화 및 취약 프로토콜 제한 39
4.15. 서버 인증서 검증 및 고정(Pinning) 41
4.16. 앱 컴포넌트(IPC) 강제 실행 및 노출 방지 42
4.17. 모바일 딥링크(DeepLink) 도용 제한 44
4.18. WebView 보안 및 JS 인터페이스 통제 45
4.19. 안전하지 않은 동적 코드 로드 방지 47
4.20. 과도한 권한(Permission) 요청 통제 48
4.21. 안전한 자격 증명(Credential) 및 API 키 관리 49
5. 별첨 52
5.1. 용어 정의 52

그림 목차

[그림 4-1] 루팅 탐지 경고 팝업 및 기능 진입 차단 10
[그림 4-2] 디코딩된 매니페스트·smali 구조 확인 10
[그림 4-3] frida-gadget 삽입 설정 10
[그림 4-4] 루팅 탐지 우회 스크립트 로드 11
[그림 4-5] 루팅 탐지 우회 후 로그인 화면 진입 성공 11
[그림 4-6] R8 난독화 활성화 설정 확인 14
[그림 4-7] proguard-rules.pro keep 규칙 확인 14
[그림 4-8] 난독화 제외 패키지 지정 내역 15
[그림 4-9] 릴리즈 APK 빌드 성공 15
[그림 4-10] mapping.txt 생성 확인 16
[그림 4-11] 난독화 릴리즈 APK 생성 확인 16
[그림 4-12] 일부 클래스 난독화 적용 확인(ApiClient→w3 등) 16
[그림 4-13] 인증 API 경로 평문 노출(@POST auth/login) 16
[그림 4-14] BASE_URL 및 API 경로 평문 노출 17
[그림 4-15] security 패키지 클래스명 원본 노출(RootDetector 등) 17
[그림 4-16] FridaDetector 클래스 구조 노출 18
[그림 4-17] frida-gadget 계측 준비 19
[그림 4-18] frida-gadget 주입 설정 20
[그림 4-19] frida-ps 대상 프로세스 확인 20
[그림 4-20] Frida attach 후 세션 정상 수립 20
[그림 4-21] 에뮬레이터 탐지 키워드 검색 0건 22
[그림 4-22] BlueStacks 프로퍼티 위장 상태 22
[그림 4-23] BlueStacks 고유 프로세스 확인 23
[그림 4-24] 앱플레이어에서 앱 정상 실행 23
[그림 4-25] 환경 프로퍼티(getprop) 확인 23
[그림 4-26] 세션 정보 암호화 저장 확인 24
[그림 4-27] SecureStore 암호화 상수 구성 25
[그림 4-28] SecureStore 키 생성·조회 로직 25

[그림 4-29] ApiClient BASE_URL 하드코딩 확인 26
[그림 4-30] 메모리 스캔을 통한 access token 평문 추출 27
[그림 4-31] logcat 수집 명령 및 HTTP 로깅 확인 29
[그림 4-32] logcat 내 로그인 비밀번호 평문 노출 29
[그림 4-33] logcat 내 AWS S3 프리사인드 URL·STS 키 노출 30
[그림 4-34] FLAG_SECURE 적용 코드 확인 32
[그림 4-35] FLAG_SECURE 상수(8192) 교차 확인 33
[그림 4-36] 멀티태스킹 미리보기 썸네일 차단 확인 35
[그림 4-37] AndroidManifest allowBackup=false 설정 확인 36
[그림 4-38] 외부 저장소 권한 미선언 및 경로 부재 확인 38
[그림 4-39] 매니페스트 usesCleartextTraffic 설정 확인 39
[그림 4-40] 실제 통신 엔드포인트 HTTPS 확인 40
[그림 4-41] 컴포넌트 exported 설정 전수 확인 43
[그림 4-42] WebView JS·브릿지 설정 및 URL 통제 부재 확인 46
[그림 4-43] 매니페스트 uses-permission 전수 확인 48
[그림 4-44] 자격증명 검색 결과 분류(하드코딩 부재) 50
[그림 4-45] persistCards 사용처 0건(카드정보 로컬 미저장) 51

표 목차

[표 1-1] 진단 대상 및 범위 1
[표 1-2] 수행 일정 1
[표 1-3] 수행 인원 및 역할 2
[표 2-1] 사전 협의 사항 3
[표 2-2] 수집 정보 및 계획 3
[표 2-3] 진단 수행 절차 3
[표 2-4] 진단 도구 목록 4
[표 2-5] 진단 기준 근거 및 출처 4
[표 2-6] 진단 항목 목록 5
[표 3-1] 진단 결과 통계 7
[표 3-2] 진단 항목별 결과 목록 7
[표 4-1] MOB-001 상세 진단 결과 9
[표 4-2] MOB-002 상세 진단 결과 12
[표 4-3] MOB-003 상세 진단 결과 13
[표 4-4] MOB-004 상세 진단 결과 19
[표 4-5] MOB-006 상세 진단 결과 22
[표 4-6] MOB-007 상세 진단 결과 24
[표 4-7] MOB-008 상세 진단 결과 26
[표 4-8] MOB-009 상세 진단 결과 27
[표 4-9] MOB-010 상세 진단 결과 29
[표 4-10] MOB-011 상세 진단 결과 31
[표 4-11] MOB-012 상세 진단 결과 34
[표 4-12] MOB-013 상세 진단 결과 36
[표 4-13] MOB-014 상세 진단 결과 38
[표 4-14] MOB-015 상세 진단 결과 39
[표 4-15] MOB-016 상세 진단 결과 41
[표 4-16] MOB-017 상세 진단 결과 42
[표 4-17] MOB-018 상세 진단 결과 44

[표 4-18] MOB-019 상세 진단 결과 45

[표 4-19] MOB-020 상세 진단 결과 47

[표 4-20] MOB-021 상세 진단 결과 48

[표 4-21] MOB-022 상세 진단 결과 49

[표 5-1] 용어 정의 52

1. 사업 개요

1.1. 사업 배경 및 목적

본 사업은 해킹의 민족(com.hackmin.app) 모바일 애플리케이션에 대하여 잠재적인 보안 취약점을 사전에 식별하고, 식별된 취약점에 대한 조치 방안을 제시하여 서비스의 보안 수준을 향상시키는 것을 목적으로 한다.

1.2. 진단 대상 및 범위

[표 1-] 진단 대상 및 범위

구분	내용
서비스명	해킹의 민족(배달 주문 앱)
진단 대상	해킹의 민족 Android Application (패키지명: com.hackmin.app)
APK 버전	[버전 1.0.0 / 빌드 번호 / SHA-256 해시]
연동 서버	https://hackmin.com (API 엔드포인트)
테스트 계정 권한	일반 사용자
진단 환경	Android AVD / BlueStacks / LDPlayer, Magisk 루팅, Frida·Burp Suite
범위 제외	APK 무결성 검증, DDoS, 사회공학 기법은 범위에서 제외

1.3. 수행 일정

[표 1-] 수행 일정

단계	세부 활동	시작일	종료일	산출물
사전 협의	대상 확정, 범위 협의, 계정 발급	2026.00.00	2026.00.00	진단 계획서

환경 구축	AVD·앱플레이어 구성, 루팅, 프리다 툴/프록시 설정	2026.07.24	2026.07.30	-
정적 분석	APK 디컴파일, 매니페스트·소스 분석	2026.07.28	2026.08.03	진단 증적
동적 분석	후킹, 트래픽 분석, 메모리 분석	2026.07.28	2026.08.03	진단 증적
결과 정리	취약점 검증, 조치 방안 도출	2026.08.01	2026.08.04	-
보고서 작성	결과 보고서 작성 및 검토	2026.08.02	2026.08.05	결과 보고서

1.4. 수행 인원 및 역할

[표 1-] 수행 인원 및 역할

성명	역할	담당 진단 항목	비고
김건하	조장 / 총괄	전체 총괄, 보고서 검수	
조건희	진단원	MOB-001 ~ MOB-008	
신승준	진단원	MOB-009 ~ MOB-015	
오현주	진단원	MOB-016 ~ MOB-022	

2. 진단 수행 방법

본 장은 해킹의 민족 모바일 앱에 대한 진단 수행 절차와 기준을 정의한다. 전자금융기반시설 모바일 보안 취약점 평가기준 및 OWASP MASTG를 기반으로 구성하였다.

2.1. 사전 협의

[표 2-] 사전 협의 사항

협의 항목	내용
진단 대상 정보	해킹의 민족 Android App (com.hackmin.app)
애플리케이션 권한	일반 사용자 계정
진단 기간	2026.07.25 ~ 2026.08.04
제약 사항	운영 서버에 부하가 갈만한 테스트 금지

2.2. 계획 수립

[표 2-] 수집 정보 및 계획

수집 항목	세부 내용
애플리케이션	대상 APK, 앱 버전 정보
서버	API 엔드포인트(https://hackmin.com), 인증 방식
계정	테스트 계정 및 권한 체계

2.3. 진단 절차

[표 2-] 진단 수행 절차

단계	절차	주요 활동
1	정보 수집	APK 획득, 패키지 구조·매니페스트 확인, 통신 흐름 파악

2	정적 분석	디컴파일을 통한 소스코드 분석, 하드코딩·설정 취약점 식별
3	동적 분석	런타임 후킹, 프록시 기반 트래픽 분석, 메모리·로컬 저장소 분석
4	취약점 검증	실제 악용 가능 여부 검증 및 증적 확보
5	결과 보고	위험도 산정, 조치 방안 도출, 결과 보고서 작성

2.4. 진단 도구

[표 2-] 진단 도구 목록

구분	도구명	버전	용도
정적 분석	JADX	[ ]	APK 디컴파일 및 소스코드 분석
정적 분석	apktool	[ ]	리소스·매니페스트 디코딩 및 리패키징
동적 분석	Frida	[ ]	런타임 후킹을 통한 로직 우회 및 메모리 분석
네트워크	Burp Suite	[ ]	프록시를 통한 요청/응답 분석 및 조작
단말 제어	adb	[ ]	로그 수집, 파일 시스템 접근, 패키지 관리
환경	Magisk	[ ]	테스트 단말 루팅 환경 구성

2.5. 진단 기준 근거 및 출처

본 진단의 항목 선정 및 양호/취약 판단 기준은 아래 문서를 근거로 한다.

[표 2-] 진단 기준 근거 및 출처

기준 문서	발행 기관	버전 / 개정일	출처

전자금융기반 시설 모바일 보안 취약점 평가 기준	금융보안원	[ ]	https://www.fsec.or.kr/bbs/detail?menuNo=301&bbsNo=6300
OWASP MASTG	OWASP	[ ]	https://mas.owasp.org/MASTG/
OWASP MASVS	OWASP	[ ]	https://mas.owasp.org/MASVS/
Android Developers Security Guidelines	Google	[ ]	https://developer.android.com/privacy-and-security/security-tips

2.6. 진단 항목

위 기준 문서의 항목 중 중요도가 '상'에 해당하는 항목을 선정하여 본 프로젝트 환경에 맞게 재구성하였다. 총 21개 항목을 진단하였다.

[표 2-] 진단 항목 목록

순번	항목명	기준	중요도
1	OS 변조(루팅/탈옥) 탐지	금취분평-7, MASTG-TEST-324	상
2	디버깅 탐지 및 차단	금취분평-16, MASTG-TEST-46	상
3	소스코드 난독화	금취분평-15, MASTG-TEST-51	상
4	동적 분석 도구 탐지	MASTG-TEST-48	상
5	보안 프로그램 및 에뮬레이터 탐지	금취분평-8·9·10, MASTG-TEST-49	상
6	단말기 내 중요정보 로컬 저장 방지	금취분평-26, MASTG-TEST-	상

		207	
7	앱 소스코드 내 운영정보 하드코딩	금취분평-43, MASTG-TEST-212	상
8	메모리 내 중요정보 노출 방지	금취분평-27	상
9	디버그 로그 내 중요정보 노출	금취분평-59, MASTG-TEST-203	상
10	화면 캡처 방지 및 중요정보 마스킹	금취분평-38, MASTG-TEST-291	상
11	백그라운드 화면 전환 시 정보 보호	금취분평-60, MASTG-TEST-289	상
12	앱 백업(AllowBackup) 설정 보안	MASTG-TEST-216	상
13	외부 저장소(External Storage) 접근 통제	MASTG-TEST-200	상
14	통신 구간 암호화 및 취약 프로토콜 제한	금취분평-65, MASTG-TEST-217	상
15	서버 인증서 검증 및 고정(Pinning)	금취분평-47, MASTG-TEST-243	상
16	앱 컴포넌트(IPC) 강제 실행 및 노출 방지	금취분평-44	상
17	모바일 딥링크(DeepLink) 도용 제한	금취분평-64	상
18	WebView 보안 및 JS 인터페이스 통제	금취분평-58, MASTG-TEST-284	상
19	안전하지 않은 동적 코드 로드 방지	Android Developers Security Checklist	상
20	과도한 권한(Permission) 요청 통제	Android Developers Security	상

		Checklist	
21	안전한 자격 증명(Credential) 및 API 키 관리	금취분평-26, MASTG-TEST-207	상

3. 진단 결과 요약

3.1. 총평

본 진단을 통해 총 21개 항목 중 7개 항목(33.3%)에서 보안 조치가 필요한 취약점이 확인되었다. 특히 디버그 로그 내 중요정보 노출(MOB-010)은 루팅 없이 adb logcat만으로 계정 비밀번호와 AWS S3 임시 자격증명이 노출되어 클라우드 자원 접근으로 직결될 수 있으므로 최우선 조치가 요구된다. 또한 루팅·동적 분석 도구 탐지 등 앱 보호 대책 계열 항목이 단일 Java 계층에 의존하여 동적 분석으로 우회 가능한 상태로 확인되었다.

3.2. 진단 결과 통계

[표 3-] 진단 결과 통계

구분	양호	취약	합계
항목 수	14	7	21
비율	66.7%	33.3%	100%

3.3. 결과 분석 및 시사점

전체 21개 항목 중 7개(33.3%)에서 취약점이 확인되었으며, 취약 항목은 대부분 앱 보호 대책(루팅·동적 분석 도구·에뮬레이터 탐지) 및 정보 노출(메모리·로그) 계열에 집중되었다. 이는 탐지 로직이 단일 지점·Java 계층에 구현되어 동적 분석 도구 한 번의 후킹으로 무력화되는 구조적 문제에 기인한다. 반면 로컬 저장·권한·컴포넌트 노출 등 기본 보안 설정은 양호하게 적용되어 있어, 클라이언트 방어 로직의 다중화·네이티브화와 서버 측 검증 병행이 핵심 개선 방향이다.

3.4. 진단 항목별 결과 목록

[표 3-] 진단 항목별 결과 목록

순번	ID	항목명	진단 결과	위험도
1	MOB-001	OS 변조(루팅/탈옥) 탐지	**취약**	상

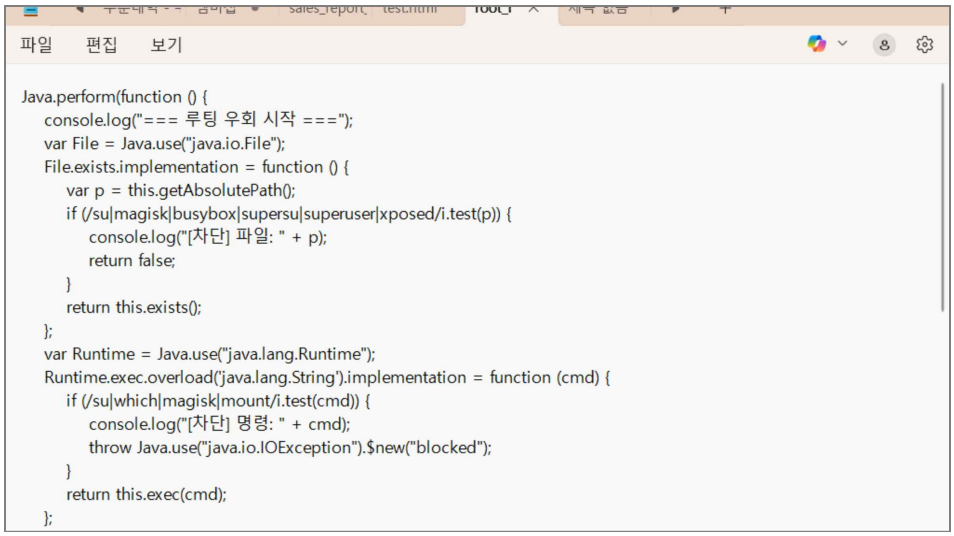
2	MOB-002	디버깅 탐지 및 차단	양호	-
3	MOB-003	소스코드 난독화	취약	상
4	MOB-004	동적 분석 도구 탐지	취약	상
5	MOB-006	보안 프로그램 및 에뮬레이터 탐지	취약	상
6	MOB-007	단말기 내 중요정보 로컬 저장 방지	양호	-
7	MOB-008	앱 소스코드 내 운영정보 하드코딩	양호	-
8	MOB-009	메모리 내 중요정보 노출 방지	취약	상
9	MOB-010	디버그 로그 내 중요정보 노출	취약	상
10	MOB-011	화면 캡처 방지 및 중요정보 마스킹	양호	-
11	MOB-012	백그라운드 화면 전환 시 정보 보호	양호	-
12	MOB-013	앱 백업(AllowBackup) 설정 보안	양호	-
13	MOB-014	외부 저장소(External Storage) 접근 통제	양호	-
14	MOB-015	통신 구간 암호화 및 취약 프로토콜 제한	양호	-
15	MOB-016	서버 인증서 검증 및 고정(Pinning)	양호	-
16	MOB-017	앱 컴포넌트(IPC) 강제 실행 및 노출 방지	양호	-
17	MOB-018	모바일 딥링크(DeepLink) 도용 제한	양호	-
18	MOB-019	WebView 보안 및 JS 인터페이스 통제	취약	상
19	MOB-020	안전하지 않은 동적 코드 로드 방지	양호	-
20	MOB-021	과도한 권한(Permission) 요청 통제	양호	-
21	MOB-022	안전한 자격 증명(Credential) 및 API 키 관리	양호	-

4. 상세 진단 결과

4.1. OS 변조(루팅/탈옥) 탐지

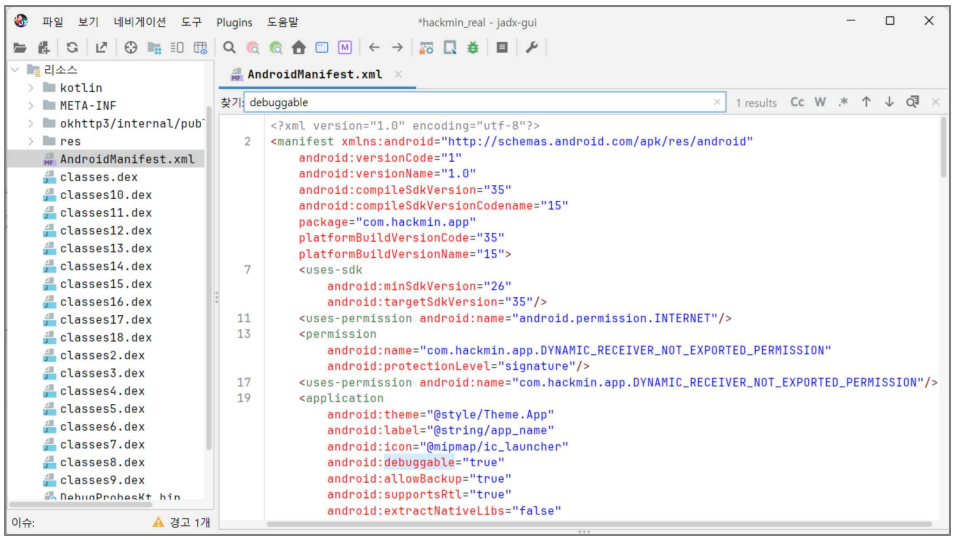
[표 4-] MOB-001 상세 진단 결과

진단 ID	MOB-001	중요도	상
진단 항목	단말기의 운영체제 변조(루팅/탈옥) 여부를 탐지하고 차단하여 비정상 환경에서의 앱 실행을 방지하는지 점검		
진단 목적	운영체제가 변조된(루팅) 단말기에서 앱이 실행될 경우, 공격자가 최고 관리자 권한을 획득하여 단말기 내 저장된 주요 정보(인증정보·금융정보 등)를 탈취하거나 메모리 변조를 통해 앱의 정상 동작을 우회할 수 있음. 따라서 이를 사전에 탐지·차단하여 보안 위협을 원천적으로 차단하기 위함.		
진단 기준	양호	단말기의 OS 변조 여부를 식별하는 로직이 적용되어 있으며, 변조가 탐지될 경우 경고 메시지 출력과 함께 앱 실행을 즉시 차단하는 경우	
	취약	OS 변조 여부를 탐지하는 로직이 없거나, 탐지하더라도 앱이 정상 실행되는 경우, 또는 루팅/탈옥 우회 도구(Magisk, Frida 등)로 탐지 로직이 쉽게 우회되는 경우	
진단 결과	취약		
진단 요약	루팅 탐지 로직은 존재하나 네이티브가 아닌 Java 계층에 구현되어 있어 동적 분석으로 우회 가능한 상태로 확인되어 취약으로 판정한다.		
진단 내용			
1) Magisk로 루팅된 환경에서 대상 앱을 최초 실행하면 "루팅된 기기에서는 앱을 이용할 수 없습니다" 경고 팝업이 노출되고 기능 진입이 차단됨을 확인하여, 탐지 로직이 존재함을 확인함.			



[그림 4-] 루팅 탐지 경고 팝업 및 기능 진입 차단

2) JADX로 대상 APK를 디컴파일하여 매니페스트 및 smali 코드에서 루팅 탐지 관련 구조를 확인함.



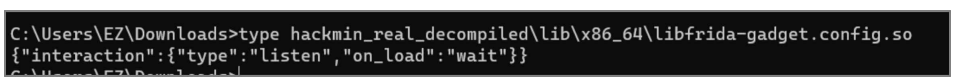
[그림 4-] 디컴파일된 매니페스트·smali 구조 확인

3) libfrida-gadget.config.so 설정을 통해 frida-gadget을 삽입하고 재빌드·재서명함.



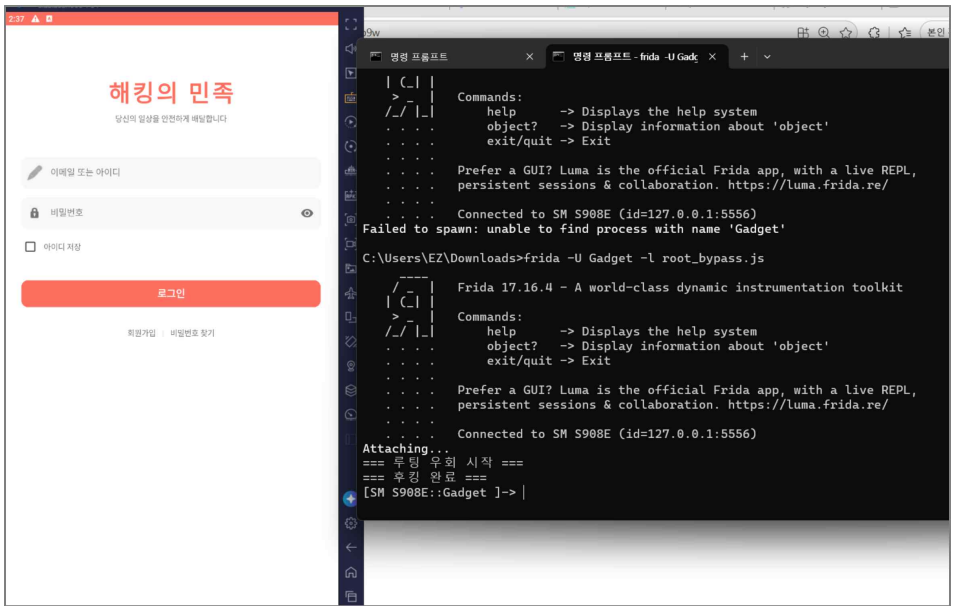
[그림 4-] frida-gadget 삽입 설정

4) root_bypass.js(File.exists / Runtime.exec에서 su·magisk·busybox 문자열 차단) 스크립트를 로드하여 탐지 로직을 우회함.



[그림 4-] 루팅 탐지 우회 스크립트 로드

5) 우회 스크립트 적용 후 앱을 재실행한 결과, 탐지 팝업 없이 로그인 화면 진입에 성공하여 탐지 로직이 무력화됨을 확인함.



[그림 4-] 루팅 탐지 우회 후 로그인 화면 진입 성공

조치 권고

현재 루팅 탐지 로직이 존재하나 Java 단일 계층에 구현되어 동적 분석으로 쉽게 무력화되므로 다음 조치를 권고함.

1) 루팅 탐지 지표 다중화: su 바이너리(/system/bin/su 등), 루팅 관리 앱 패키지(Magisk/SuperSU), Build.TAGS의 test-keys, 시스템 파티션 쓰기 가능 여부, ro.debuggable·ro.secure 등 복수 지표를 함께 검사할 것.

2) 탐지 로직을 네이티브(C/C++) 계층으로 이전하고, 무결성 검증·후킹 탐지와 병행하여 Frida 우회 난이도를 확보할 것.

3) 탐지 시 즉시 종료보다 주요 기능(로그인·결제) 제한 후 서버 통보 방식을 권장함.

4.2. 디버깅 탐지 및 차단

[표 4-] MOB-002 상세 진단 결과

진단 ID	MOB-002	중요도	상
진단 항목	앱이 디버거 연결을 탐지·차단하여 런타임 디버깅을 통한 로직 분석·변조를 방지하는지 점검		
진단 목적	디버거가 연결된 상태에서 앱이 실행되면 공격자가 실행 흐름을 단계별로 추적·변조하여 인증 우회, 로직 조작 등이 가능함. 따라서 디버깅 가능 상태를 탐지·차단하는지 점검함.		
진단 기준	양호	release 빌드에서 debuggable 플래그가 비활성화되어 있고, 디버거 연결을 탐지·차단하는 경우	
	취약	release 빌드에서 debuggable이 활성화되어 있거나 디버거 연결을 탐지하지 못하는 경우	
진단 결과	양호		
진단 요약	debuggable 플래그는 debug 빌드에서 자동으로 부여되는 값이며, release 빌드 기준으로는 비활성화되어 양호로 판정한다.		
진단 내용			
1) release 빌드 기준으로 debuggable 플래그 상태를 확인한 결과 디버깅이 허용되지 않음을 확인함.			
조치 권고			
해당 사항 없음.			

4.3. 소스코드 난독화

[표 4-] MOB-003 상세 진단 결과

진단 ID	MOB-003	중요도	상
진단 항목	R8/ProGuard 등 난독화가 적용되어 클래스·메서드·문자열 등 핵심 식별자가 분석으로부터 보호되는지 점검		
진단 목적	난독화가 적용되지 않으면 디컴파일만으로 소스 수준의 로직·인증 흐름·보안 로직이 그대로 노출되어 공격 지점 특정이 용이해짐. 따라서 식별자 치환 및 보안 로직 보호 여부를 점검함.		
진단 기준	양호	핵심 클래스·메서드·필드명이 난독화(a, b, c 등)되어 있고, 보안 관련 로직과 문자열이 평문으로 노출되지 않는 경우	
	취약	주요 클래스·메서드·경로 문자열이 원본 그대로 노출되거나, keep 규칙 등으로 보안·인증 관련 패키지가 난독화에서 제외되는 경우	
진단 결과	취약		
진단 요약	R8 난독화는 활성화되어 있으나 keep 규칙으로 data.api·data.model·security 패키지가 제외되어 인증 흐름과 보안 탐지 로직이 평문으로 노출되므로 취약으로 판정한다.		

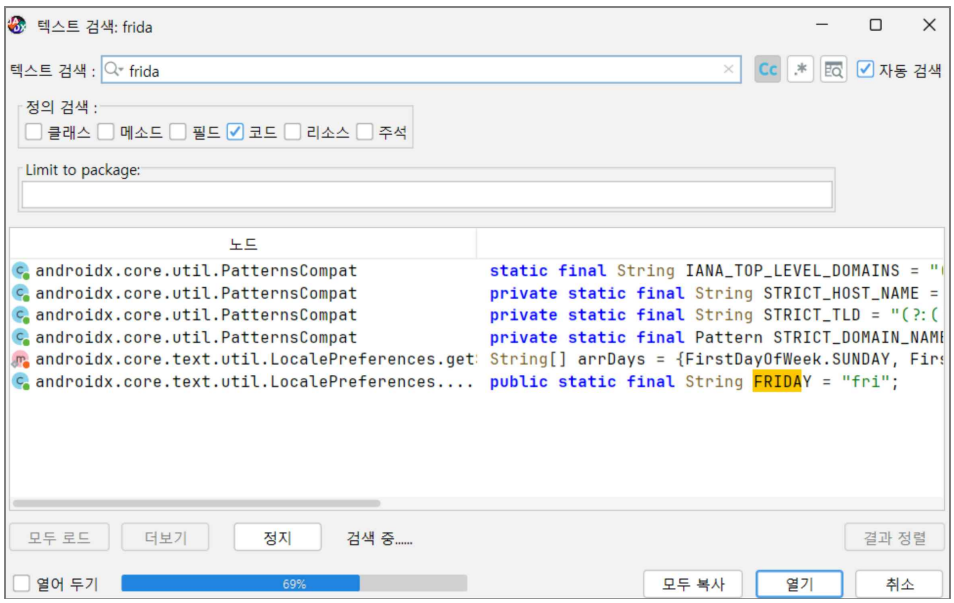
진단 내용

1) JADX로 릴리즈 apk를 디컴파일 한 결과, R8난독화가 적용되어 있지만, API, DTO 클래스 파일들은 난독화가 되어있지 않음

```
/* JADX INFO: compiled from: r8-map-id-e8bff171bebf3bc659569ecb1b28619f6e9c7e742
/* JADX INFO: loaded from: classes.dex */
public class CartActivity extends z7 {
    public static final /* synthetic */ int J = 0;
    public TextView A;
    public TextView B;
    public TextView C;
    public TextView D;
    public Button E;
    public y7 F;
    public CartApi G;
    public final NumberFormat H = NumberFormat.getNumberInstance(Locale.KOREA);
    public CartSummaryDto I;
    public RecyclerView z;
```

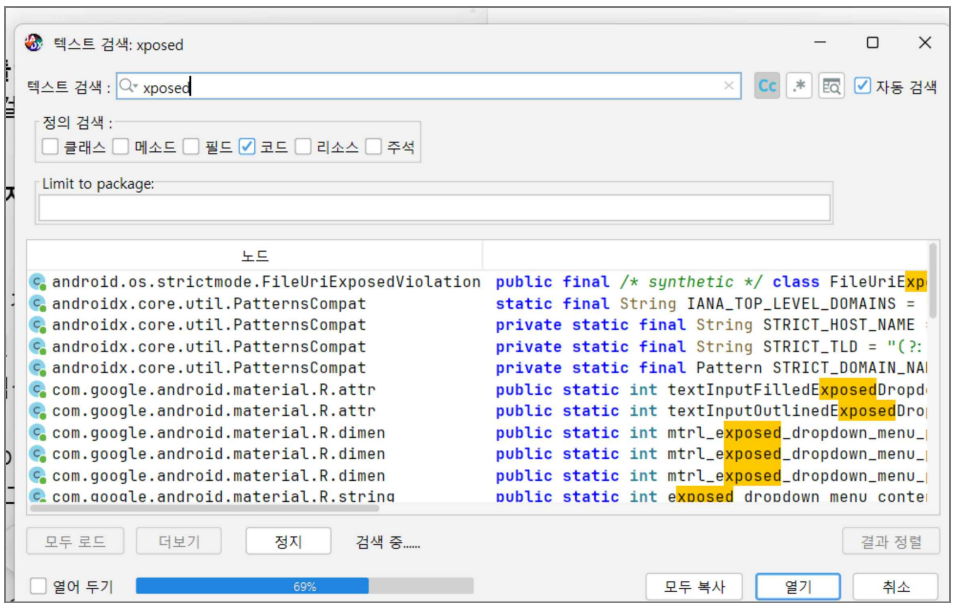
[그림 4-] R8 난독화 활성화 설정 확인

2) 다만 data.api·data클래스들이 난독화 대상에서 제외되어 있음을 확인함.



[그림 4-] progūard-rules.pro keep 규칙 확인

3) keep 규칙 상세 — 보안·모델 패키지가 난독화 제외 대상으로 지정됨.



[그림 4-] 난독화 제외 패키지 지정 내역

4) 위 구성으로 릴리즈 APK를 빌드하여 BUILD SUCCESSFUL을 확인함.

```
C:\Users\EZ\Downloads>java -jar apktool_3.0.3.jar d hackmin_real.apk -o hackmin_real_decompiled -f
I: Using Apktool 3.0.3 on hackmin_real.apk with 8 threads
I: Baksmaling classes10.dex...
I: Baksmaling classes8.dex...
I: Baksmaling classes4.dex...
I: Loading resource table...
I: Baksmaling classes3.dex...
I: Baksmaling classes16.dex...
I: Baksmaling classes12.dex...
I: Baksmaling classes13.dex...
I: Baksmaling classes11.dex...
I: Baksmaling classes6.dex...
I: Decoding value resources...
I: Loading resource table from file: C:\Users\EZ\AppData\Local\apktool\framework\1.apk
I: Baksmaling classes9.dex...
I: Baksmaling classes14.dex...
I: Baksmaling classes17.dex...
I: Baksmaling classes7.dex...
I: Baksmaling classes5.dex...
I: Baksmaling classes15.dex...
I: Baksmaling classes2.dex...
I: Baksmaling classes18.dex...
I: Baksmaling classes.dex...
I: Decoding file resources...
I: Generating values XMLs...
I: Decoding AndroidManifest.xml with resources...
I: Copying original files...
I: Copying unknown files...

C:\Users\EZ\Downloads>findstr "debuggable" hackmin_real_decompiled\AndroidManifest.xml
    <application android:allowBackup="true" android:appComponentFactory="androidx.core.app.CoreComponentFactory" android
:dataExtractionRules="@xml/data_extraction_rules" android:debuggable="true" android:extractNativeLibs="false" android:fu
llBackupContent="@xml/backup_rules" android:icon="@mipmap/ic_launcher" android:label="@string/app_name" android:roundIco
n="@mipmap/ic_launcher_round" android:supportsRtl="true" android:theme="@style/Theme.App" android:usesCleartextTraffic="
true">
```

[그림 4-] 릴리즈 APK 빌드 성공

5) 빌드 결과 난독화 매핑 파일(mapping.txt, 약 13.6MB)이 정상 생성됨을 확인함.

```
C:\Users\EZ\Downloads>mkdir hackmin_real_decompiled\lib\x86_64

C:\Users\EZ\Downloads>copy /Y "C:\Users\EZ\Downloads\frida-gadget-17.16.4-android-x86_64.so\frida-gadget-17.16.4-android
-x86_64.so" hackmin_real_decompiled\lib\x86_64\libfrida-gadget.so
        1개 파일이 복사되었습니다.

C:\Users\EZ\Downloads>dir hackmin_real_decompiled\lib\x86_64\
 C 드라이브의 볼륨에는 이름이 없습니다.
 볼륨 일련 번호: 66D7-DF83

 C:\Users\EZ\Downloads\hackmin_real_decompiled\lib\x86_64 디렉터리

2026-07-28  오후 06:09    <DIR>          .
2026-07-28  오후 06:09    <DIR>          ..
2026-07-28  오전 11:22        26,590,424 libfrida-gadget.so
               1개 파일          26,590,424 바이트
               2개 디렉터리   6,053,007,360 바이트 남음

C:\Users\EZ\Downloads>dir /s /b C:\Users\EZ\Downloads\frida-gadget*x86_64.so
C:\Users\EZ\Downloads\frida-gadget-17.16.4-android-x86_64.so
C:\Users\EZ\Downloads\frida-gadget-17.16.4-android-x86_64.so\frida-gadget-17.16.4-android-x86_64.so
```

[그림 4-] mapping.txt 생성 확인

6) 난독화된 릴리즈 APK(app-release-unsigned.apk)가 정상 생성됨을 확인함.

```
C:\Users\EZ\Downloads>powershell -Command "$f='hackmin_real_decompiled\AndroidManifest.xml'; $c=[System.IO.File]::ReadAl
lText($f); $c=$c.Replace('android:extractNativeLibs=\"false\"','android:extractNativeLibs=\"true\"'); [System.IO.File]::
WriteAllText($f,$c,[System.Text.UTF8Encoding]::new($false))"
```

[그림 4-] 난독화 릴리즈 APK 생성 확인

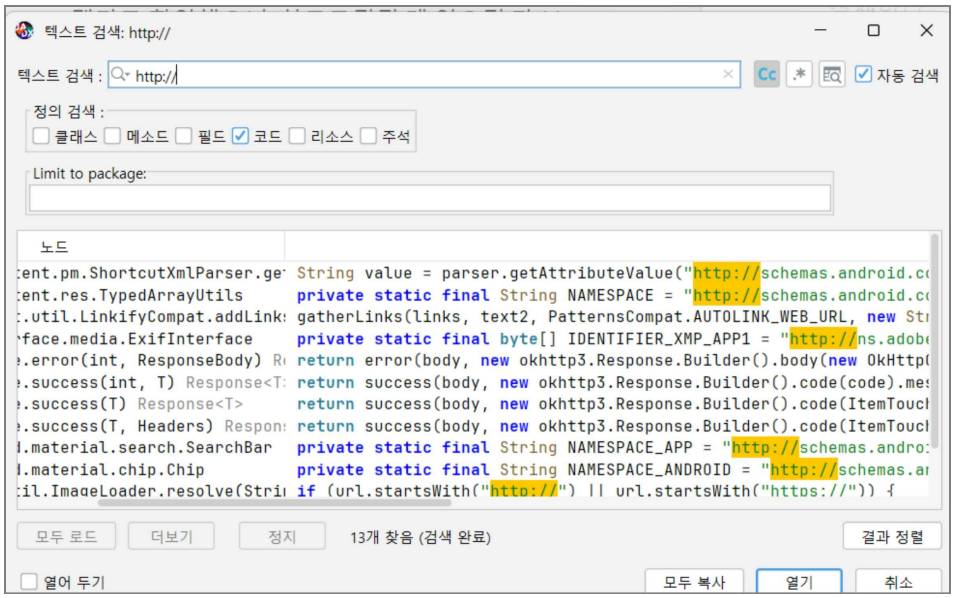
7) 매핑 파일 분석 결과 ApiClient→w3, SessionManager→w70 등 일부 클래스는 실제로 난독화가 적용되었음을 확인함.

```
C:\Users\EZ\Downloads>findstr "extractNativeLibs" hackmin_real_decompiled\AndroidManifest.xml
    <application android:allowBackup="true" android:appComponentFactory="androidx.core.app.CoreComponentFactory" android
:dataExtractionRules="@xml/data_extraction_rules" android:debuggable="true" android:extractNativeLibs="true" android:ful
lBackupContent="@xml/backup_rules" android:icon="@mipmap/ic_launcher" android:label="@string/app_name" android:roundIcon
="@mipmap/ic_launcher_round" android:supportsRtl="true" android:theme="@style/Theme.App" android:usesCleartextTraffic="t
rue">
```

[그림 4-] 일부 클래스 난독화 적용 확인(ApiClient→w3 등)

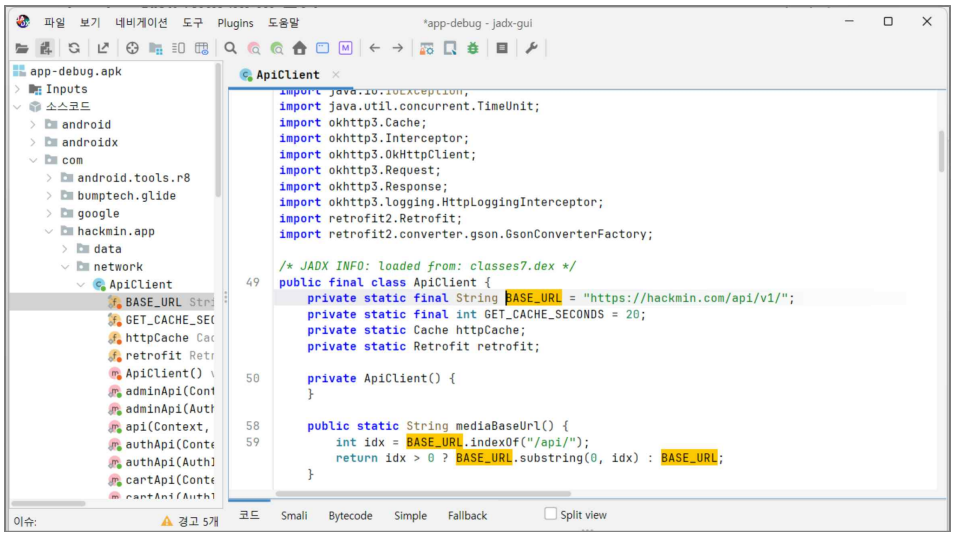
8) 그러나 난독화된 APK를 JADX로 디컴파일한 결과, 인증 API 인터페이스와 엔드포인트

경로(@POST("auth/login") 등)가 평문으로 노출되어 인증 흐름을 그대로 추적할 수 있었음.



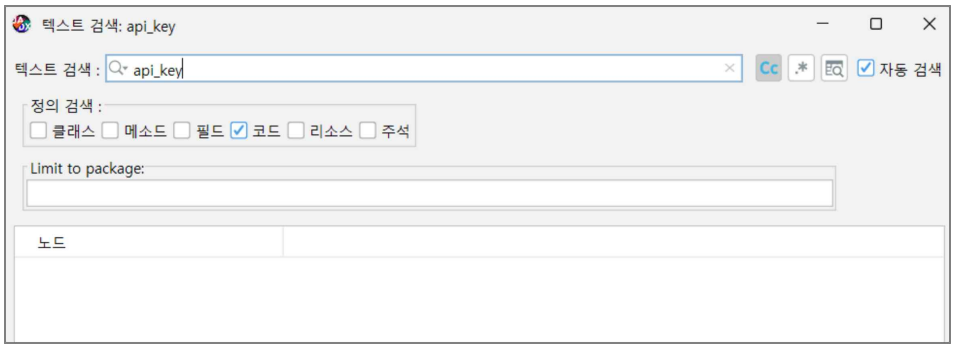
[그림 4-] 인증1API 경로 평문 노출(@POST auth/login)

9) ApiClient의 BASE_URL 등 운영 정보도 평문으로 확인되어 난독화된 메서드의 실제 기능이 경로 문자열로 드러남.



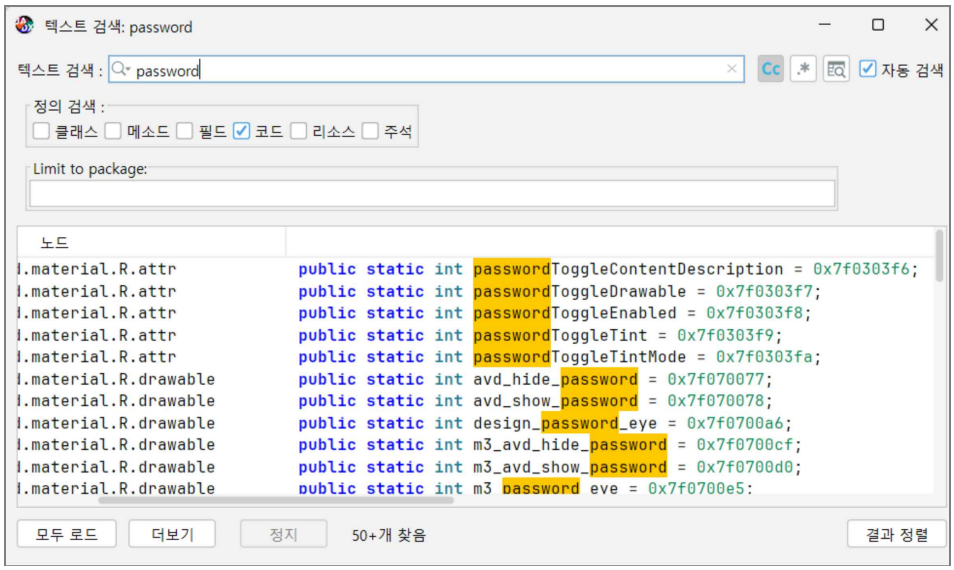
[그림 4-] BASE4URL 및 API 경로 평문 노출

10) 보안 탐지 로직(security 패키지)의 클래스명이 원본 그대로 노출되어, RootDetector(루팅 탐지)·FridaDetector(동적 분석 탐지)·SecureStore(보안 저장소) 등 클래스명만으로 어떤 보안 로직이 구현되어 있는지 즉시 식별 가능함.



[그림 4-] security 패키지 클래스명 원본 노출(RootDetector 등)

11) 동일하게 FridaDetector 등 탐지 클래스 구조가 노출되어 탐지 우회 지점을 특정하는 데 악용될 수 있음.



[그림 4-] FridaDetector 클래스 구조 노출

조치 권고

R8은 식별자 치환 수준이며 문자열 상수는 노출되므로 다음을 권고함.

1) 문자열 암호화·제어흐름 난독화는 상용 도구 영역으로, 핵심 보안 로직은 NDK 네이티브(.so) 계층에 구현할 것.

2) keep 규칙으로 인한 보안·인증 패키지 난독화 제외를 재검토하여 최소한으로 축소할 것.

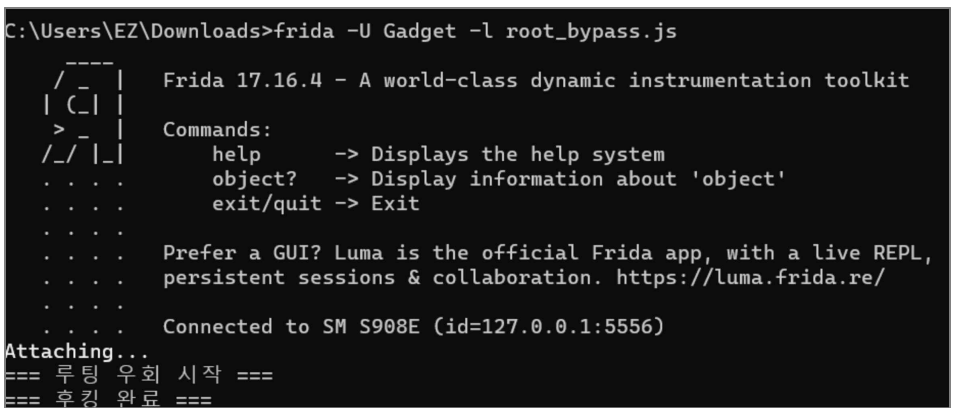
3) 난독화는 흐름 노출을 늦출 뿐 근본 차단이 아니므로, 흐름이 노출되어도 공격이 성립하지 않도록 서버 측 방어를 병행할 것.

4.4. 동적 분석 도구 탐지

[표 4-] MOB-004 상세 진단 결과

진단 ID	MOB-004	중요도	상
진단 항목	Frida 등 동적 분석 도구의 실행·연결을 탐지·차단하여 런타임 후킹을 방지하는지 점검		
진단 목적	Frida 등 후킹 도구가 제약 없이 동작하면 인증·보안 로직을 런타임에 무력화할 수 있음. 따라서 동적 분석 도구의 탐지·차단 여부를 점검함.		
진단 기준	양호	Frida 등 도구의 프로세스·포트·메모리 특징을 탐지하고, 탐지 시 앱 실행을 차단하며 탐지 지점이 코드에 노출되지 않는 경우	
	취약	탐지 로직이 없거나, 탐지하더라도 판별 지점이 평문 노출되어 Java 계층 후킹으로 쉽게 무력화되는 경우	
진단 결과	취약		
진단 요약	Frida 탐지 관련 로직이 존재하더라도 판별 메서드·상수가 앱 코드에 평문 노출되고 Java 계층에 위치하여 동일 프로세스 내 후킹으로 무력화 가능한 자기무효화 상태이므로 취약으로 판정한다.		
진단 내용			

1) frida-gadget을 삽입한 상태에서 계측 세션을 준비함.



[그림 4-] frida-gadget 계측 준비

2) gadget 설정(libfrida-gadget.config.so)을 통해 앱 프로세스에 Frida를 주입함.

```
C:\Users\EZ\Downloads>frida -U Gadget -l root_bypass.js
     ____
    / _  |   Frida 17.16.4 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   Prefer a GUI? Luma is the official Frida app, with a live REPL,
   . . . .   persistent sessions & collaboration. https://luma.frida.re/
   . . . .
   . . . .   Connected to SM S908E (id=127.0.0.1:5556)
Attaching...
=== 루팅 우회 시작 ===
=== 후킹 완료 ===
[SM S908E::Gadget ]->
```

[그림 4- 18] frida-gadget 주입 설정

3) frida-ps로 대상 앱 프로세스를 확인함.

```
C:\Users\EZ>frida-ps -D emulator-5554 | findstr app
]4201  app
2277  com.google.process.gapps
```

[그림 4- 19] frida-ps 대상 프로세스 확인

4) frida -U -f com.hackmin.app(spawn) 및 attach로 계측을 시도한 결과 탐지·경고·종료 없이 세션이 정상 수립(Java.available → true)되고 후킹이 제약 없이 가능함을 확인함.

```
C:\Users\EZ>frida -D emulator-5554 -n app
     ____
    / _  |   Frida 17.16.4 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   Prefer a GUI? Luma is the official Frida app, with a live REPL,
   . . . .   persistent sessions & collaboration. https://luma.frida.re/
   . . . .
   . . . .   Connected to Android Emulator 5554 (id=emulator-5554)

[Android Emulator 5554::app ]->
```

[그림 4- 20] Frida attach 후 세션 정상 수립

조치 권고

탐지 로직이 존재하더라도 판별 지점이 노출되고 Java 계층에 위치하여 무력화 가능하므로 다음을 권고함.

1) 판별 지점(포트·프로세스명·경로 문자열)을 코드에 평문 노출하지 말고 암호화하며, 탐지 메서드는 난독화로 이름을 은닉할 것.
2) 탐지 사실을 logcat에 남기지 말 것(로그가 우회 지도가 됨).
3) 탐지 로직을 네이티브(JNI/.so)로 이전하여 Java 후킹으로 반환값을 바꾸지 못하게 할 것.
4) 단일 boolean 반환을 지양하고 여러 지점에서 분산 검사하여 훅 하나로 전체가 꺼지지 않게 할 것.
5) 서버 측 검증(Play Integrity)·무결성 검증을 병행하고, 탐지 시 즉시 종료보다 서버 통보 후 민감 기능을 차단할 것.

4.5. 보안 프로그램 및 에뮬레이터 탐지

[표 4-] MOB-006 상세 진단 결과

진단 ID	MOB-006	중요도	상
진단 항목	에뮬레이터·앱플레이어 등 가상 환경에서의 실행을 탐지·차단하는지 점검		
진단 목적	가상 환경에서 앱이 제약 없이 실행되면 자동화된 대량 분석·조작에 노출됨. 따라서 에뮬레이터·앱플레이어 탐지 여부를 점검함.		
진단 기준	양호	프로세스·파일시스템·하드웨어 특성 등 복수 지표로 가상 환경을 탐지하고 실행을 제한하는 경우	
	취약	탐지 로직이 없거나 프로퍼티 위장으로 통상 탐지가 무력화되는 경우	
진단 결과	취약		
진단 요약	에뮬레이터 탐지 로직이 미구현되어 BlueStacks·AVD·LDPlayer 모두에서 설치부터 결제까지 제한 없이 수행되므로 취약으로 판정한다.		

진단 내용

1) JADX에서 에뮬레이터 탐지 키워드(Build.FINGERPRINT/MODEL, emulator, goldfish, genymotion, Xposed 등)를 전수 검색한 결과 0건으로, 탐지 로직이 미구현됨을 확인함.

```
C:\Users\EZ>findstr /s /i "goldfish ranchu qemu bluestacks genymotion generic 15555215554 isEmulator" hackmin_real_decompiled\smali*\com\hackmin\*.smali

C:\Users\EZ>
```

[그림 4-] 에뮬레이터 탐지 키워드 검색 0건

2) BlueStacks가 단말 프로퍼티를 실기기로 위장(release-keys 서명, ro.kernel.qemu 미설정)하여 통상적인 FINGERPRINT/qemu 기반 탐지가 무력화됨을 확인함.

```
[Android Emulator 5554::app ]-> Process.enumerateRanges('rw-').forEach(function(r){try{Memory.scanSync(r.base,r.size,"6
f 79 4a 68 62 47 63 69").forEach(function(m){try{console.log("[토큰발견] "+m.address+": "+Memory.readUtf8String(m.addre
ss,120));}catch(e){}});}catch(e){}});
[Android Emulator 5554::app ]-> function findStr(s){var h=s.split('').map(function(c){return c.charCodeAt(0).toString(1
6).padStart(2,'0');}).join(' ');var n=0;Process.enumerateRanges('rw-').forEach(function(r){try{Memory.scanSync(r.base,r
.size,h).forEach(function(m){n++;if(n<=5)try{console.log("[발견] "+m.address+": "+Memory.readUtf8String(m.address,60));
}catch(e){}});}catch(e){}});console.log(s+" → "+n+"건");}
[Android Emulator 5554::app ]-> findStr("kkkk@gmail.com")
kkkk@gmail.com → 14건
[Android Emulator 5554::app ]-> findStr("로그인비번")
로그인비번 → 0건
[Android Emulator 5554::app ]->
```

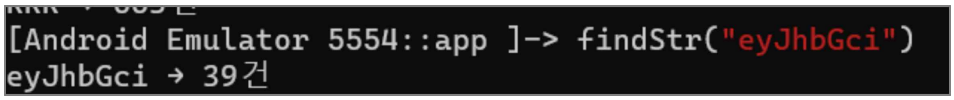
[그림 4-] BlueStacks 프로퍼티 위장 상태

3) 다만 프로세스 목록에서 bstsvcmgrtest·com.bluestacks.BstCommandProcessor 등 앱플레이어 고유 프로세스를 확인함.



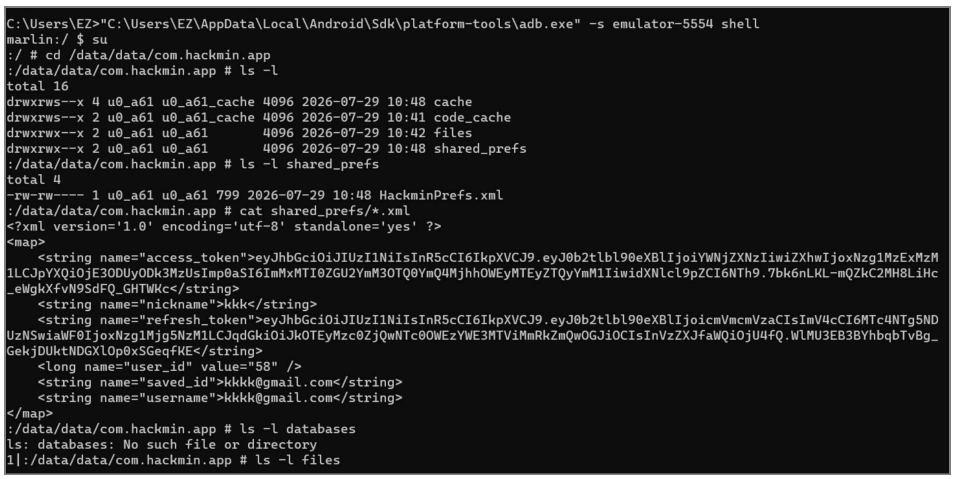
[그림 4-] BlueStacks 고유 프로세스 확인

4) 앱플레이어 환경에서 앱이 정상 실행되어 로그인·결제까지 제한 없이 수행됨을 확인함.



[그림 4-] 앱플레이어에서 앱 정상 실행

5) getprop 등으로 환경 프로퍼티를 확인하여 환경별 탐지 난이도 차이를 확인함(AVD는 qemu-props·ranchu 등 확인됨).



[그림 4-] 환경 프로퍼티(getprop) 확인

조치 권고

프로퍼티 단독으로는 상용 앱플레이어 탐지가 어려우므로 다음을 권고함.

1) 다중 지표 탐지: 프로세스명(bst*, com.bluestacks.*, nox, ldplayer), 파일시스템(/dev/qemu_pipe, /dev/socket/qemud), 하드웨어 특성(센서·통신사 부재), CPU 아키텍처 불일치 등을 종합 검사할 것.

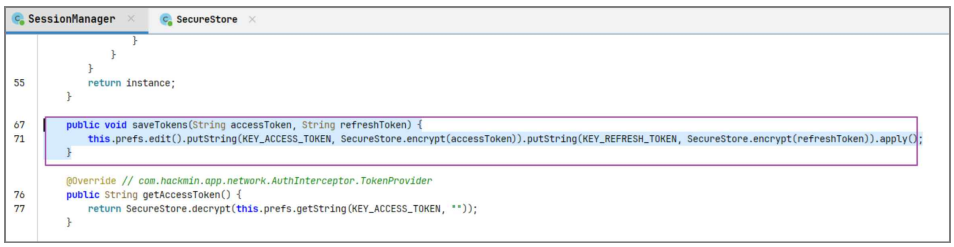
2) Play Integrity API로 서버 측 무결성 검증을 병행할 것.

4.6. 단말기 내 중요정보 로컬 저장 방지

[표 4-] MOB-007 상세 진단 결과

진단 ID	MOB-007	중요도	상
진단 항목	인증정보·개인정보 등 중요 데이터가 단말기 내에 안전하게(암호화) 저장되는지 점검		
진단 목적	중요정보가 평문으로 로컬 저장되면 단말 탈취·루팅 시 그대로 유출됨. 따라서 로컬 저장 데이터의 암호화 여부를 점검함.		
진단 기준	양호	인증 토큰 등 중요정보를 Android Keystore 기반 암호화 저장소에 저장하고, 카드번호·비밀번호 등을 로컬에 평문 저장하지 않는 경우	
	취약	인증 토큰·개인정보가 비암호화 SharedPreferences 등에 평문으로 저장되는 경우	
진단 결과	양호		
진단 요약	중요정보를 Android Keystore 기반 암호화 저장소(SecureStore)에 저장하고, 카드번호·비밀번호는 로컬에 저장하지 않음을 확인하여 양호로 판정한다.		
진단 내용			

1) 세션 관련 데이터가 암호화되어 저장됨을 확인함.



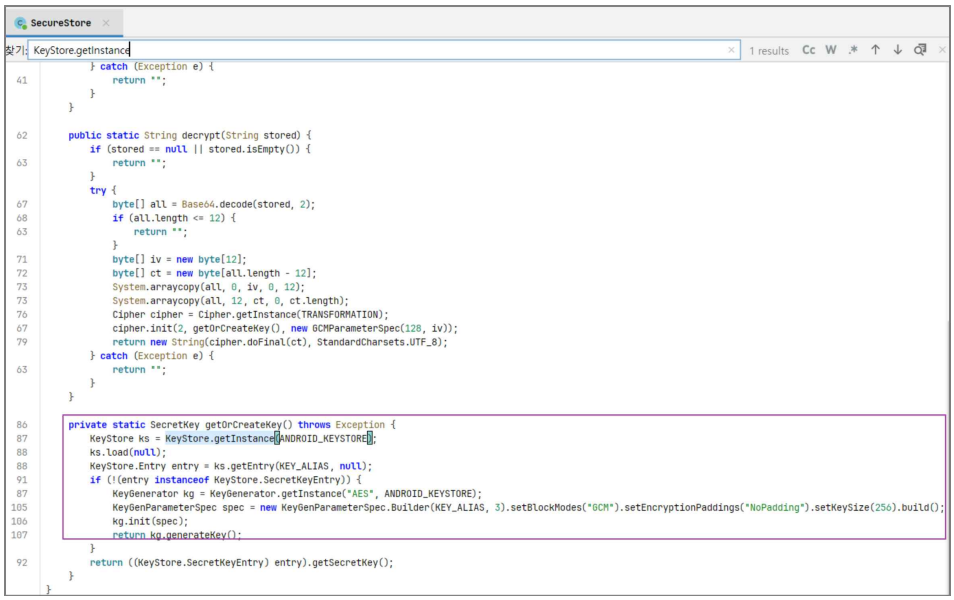
[그림 4-] 세션 정보 암호화 저장 확인

2) SecureStore가 Android Keystore 기반 암호화 상수(AES/GCM 등)를 사용하도록 구성됨을 확인함.



[그림 4-] SecureStore 암호화 상수 구성

3) SecureStore.getOrCreateKey()가 Keystore에서 키를 생성·조회하여 저장 데이터를 암호화함을 확인함.



[그림 4-] SecureStore 키 생성·조회 로직

조치 권고

해당 사항 없음.

4.7. 앱 소스코드 내 운영정보 하드코딩

[표 4-] MOB-008 상세 진단 결과

진단 ID	MOB-008	중요도	상
진단 항목	소스코드 내에 운영 서버 주소·자격증명 등 민감 운영정보가 하드코딩되어 있는지 점검		
진단 목적	운영정보가 하드코딩되면 디컴파일만으로 인프라 구조·접근 정보가 노출됨. 따라서 민감 운영정보의 하드코딩 여부를 점검함.		
진단 기준	양호	민감한 자격증명·키 등이 소스에 하드코딩되지 않은 경우(서버 주소 노출은 일반적이나 자격증명 노출이 없어야 함)	
	취약	API 키·시크릿·자격증명 등 민감 정보가 소스에 평문 하드코딩된 경우	
진단 결과	양호		
진단 요약	ApiClient의 BASE_URL에 운영 서버 주소가 평문으로 하드코딩되어 있으나 그 외 민감한 자격증명·키의 하드코딩은 발견되지 않아 양호로 판정한다.		

진단 내용

1) JADX로 ApiClient를 확인한 결과 BASE_URL에 운영 서버 주소가 평문으로 하드코딩되어 있으나, 그 외 API 키·시크릿 등 민감 정보의 하드코딩은 발견되지 않음.

```
/* JADX INFO: loaded from: classes7.dex */
public final class ApiClient {
    private static final String BASE_URL = "https://hackmin.com/api/v1/";
    private static final int GET_CACHE_SECONDS = 20;
```

[그림 4-] ApiClient BASE_URL 하드코딩 확인

조치 권고

민감 자격증명 하드코딩은 없으나 하드닝 관점에서 다음을 권고함.

1) 서버 주소는 IP 대신 도메인을 사용하고, 향후 키·시크릿 도입 시 소스 하드코딩을 지양할 것.

4.8. 메모리 내 중요정보 노출 방지

[표 4-] MOB-009 상세 진단 결과

진단 ID	MOB-009	중요도	상
진단 항목	인증 토큰 등 중요정보가 런타임 메모리에 평문으로 잔존하여 추출 가능한지 점검		
진단 목적	토큰이 메모리에 평문으로 상주하면 메모리 덤프·인젝션으로 추출되어 세션 하이재킹이 가능함. 따라서 메모리 내 중요정보 노출 여부를 점검함.		
진단 기준	양호	중요정보를 사용 직후 즉시 소거하고, 힙·메모리 스캔으로 평문 자격증명이 추출되지 않는 경우	
	취약	힙 객체 조회 또는 메모리 스캔으로 access token 등 평문 자격증명이 추출되는 경우	
진단 결과	취약		
진단 요약	Frida로 힙 내 SessionManager 인스턴스를 조회하고 메모리 전역 스캔으로 access token(JWT)을 평문 추출할 수 있어 취약으로 판정한다.		

진단 내용

1) 로그인 후 세션 유지 상태에서 frida attach 후 Java.choose로 힙 내 SessionManager 싱글톤 인스턴스를 획득하고, Process.enumerateRanges + Memory.scanSync로 access token(JWT) 헤더 문자열을 메모리 전역에서 검색하여 다수 주소에서 평문 추출에 성공함.

```
[Android Emulator 5554::app ]-> whereStr("kkkk@gmail.com")
[Android Emulator 5554::app ]-> function whereStr(s){var h=s.split('').map(function(c){return c.charCodeAt(0).
16).padStart(2,'0');}).join(' ');var n=0;Process.enumerateRanges('rw-').forEach(function(r){try{Memory.scanSyn
r.size,h).forEach(function(m){n++;console.log("["+n+"] 주소 "+m.address+" | 영역 "+r.base+"~"+r.base.add(r.siz
.protection+")");});}catch(e){}});console.log("총 "+n+"곳 발견");}
[Android Emulator 5554::app ]-> whereStr("kkkk@gmail.com")
[1] 주소 0x12eef9f0 | 영역 0x12c00000~0x13900000 (rw-)
[2] 주소 0x12eefb20 | 영역 0x12c00000~0x13900000 (rw-)
[3] 주소 0x130736d8 | 영역 0x12c00000~0x13900000 (rw-)
[4] 주소 0x76386705d2fe | 영역 0x763863e00000~0x763869000000 (rw-)
[5] 주소 0x76386705d317 | 영역 0x763863e00000~0x763869000000 (rw-)
[6] 주소 0x7638718b0043 | 영역 0x7638711b0000~0x7638731b0000 (rw-)
[7] 주소 0x7638757d121d | 영역 0x763875000000~0x763875800000 (rw-)
[8] 주소 0x7638757d1236 | 영역 0x763875000000~0x763875800000 (rw-)
[9] 주소 0x76388891b975 | 영역 0x763888800000~0x763888a00000 (rw-)
[10] 주소 0x76388891b98e | 영역 0x763888800000~0x763888a00000 (rw-)
[11] 주소 0x76388be1c355 | 영역 0x76388be00000~0x76388c000000 (rw-)
[12] 주소 0x76388be1c36e | 영역 0x76388be00000~0x76388c000000 (rw-)
[13] 주소 0x76388bf5ed02 | 영역 0x76388be00000~0x76388c000000 (rw-)
[14] 주소 0x76388bf5ed1b | 영역 0x76388be00000~0x76388c000000 (rw-)
총 14곳 발견
```

[그림 4-] 메모리 스캔을 통한 access token 평문 추출

조치 권고
저장소 암호화와 무관하게 토큰은 요청 헤더 첨부를 위해 런타임 메모리에 평문 존재하므로 다음을 권고함. 1) 사용 직후 자격증명 객체를 즉시 null 처리·GC 유도하고, 민감 문자열은 char[]로 다루어 사용 후 즉시 소거할 것(String 불변성 잔존 방지). 2) 근본적으로 후킹 탐지를 선행하여 메모리 접근 자체를 어렵게 할 것.

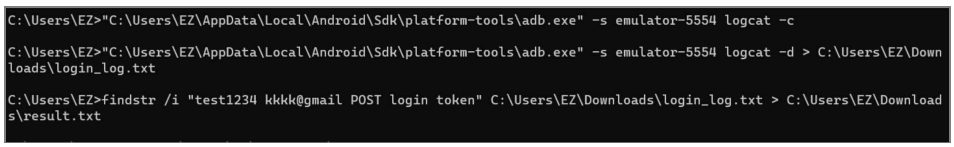
4.9. 디버그 로그 내 중요정보 노출

[표 4-] MOB-010 상세 진단 결과

진단 ID	MOB-010	중요도	상
진단 항목	디버그 로그(logcat)에 비밀번호·토큰·자격증명 등 중요정보가 기록되는지 점검		
진단 목적	로그에 민감정보가 남으면 루팅 없이 logcat만으로 계정·클라우드 자원이 노출됨. 따라서 로그 내 중요정보 노출 여부를 점검함.		
진단 기준	양호	release 빌드에서 HTTP 본문·자격증명이 로그에 기록되지 않는 경우	
	취약	요청/응답 본문 전체가 로그에 기록되어 비밀번호·토큰·서명 URL 등이 평문 노출되는 경우	
진단 결과	취약		
진단 요약	HttpLoggingInterceptor가 Level.BODY로 설정되어 로그인 비밀번호 평문과 AWS S3 임시 자격증명(STS 키)이 logcat에 노출되므로 최우선 조치 대상으로 취약 판정한다.		

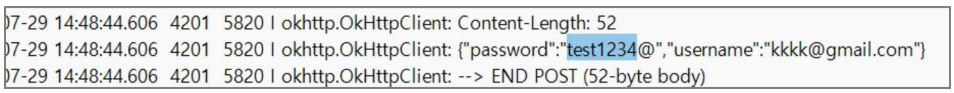
진단 내용

1) adb logcat -c로 로그를 초기화한 뒤 재로그인하며 로그를 캡처함. HttpLoggingInterceptor 레벨이 Level.BODY로 설정되어 okhttp 태그로 모든 HTTP 통신이 로그에 기록됨을 확인함.



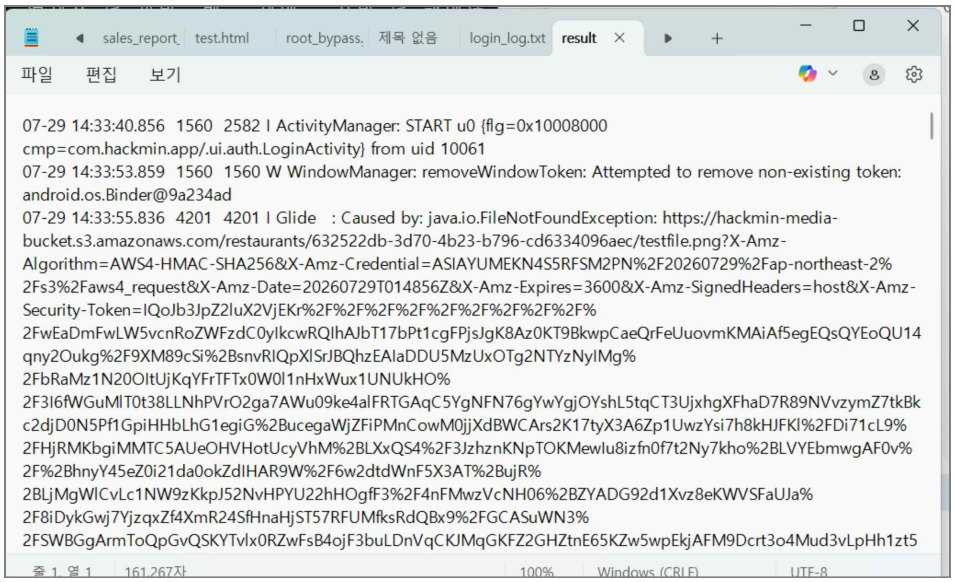
[그림 4-] logcat 수집 명령 및 HTTP 로깅 확인

2) 재로그인 시 요청 본문에 계정 비밀번호가 평문으로 기록됨을 확인함.



[그림 4-] logcat 내 로그인 비밀번호 평문 노출

3) 이미지/첨부 프리사인드 URL이 로그에 남으며 AWS S3 임시 자격증명(X-Amz-Credential=ASIA..., 서명)이 그대로 노출되어, 해당 URL로 약 1시간 동안 S3 버킷 객체 접근이 가능함을 확인함.



[그림 4-] logcat 내 AWS S3 프리사인드 URL·STS 키 노출

조치 권고

루팅 없이 adb logcat만으로 계정 정보와 클라우드 자원이 함께 노출되므로 즉시 조치가 필요함.

1) (즉시) 노출된 AWS 자격증명/프리사인드 URL을 무효화하고 발급 정책을 점검할 것 — STS 토큰 수명 단축, 클라이언트에 장기 키 미노출.

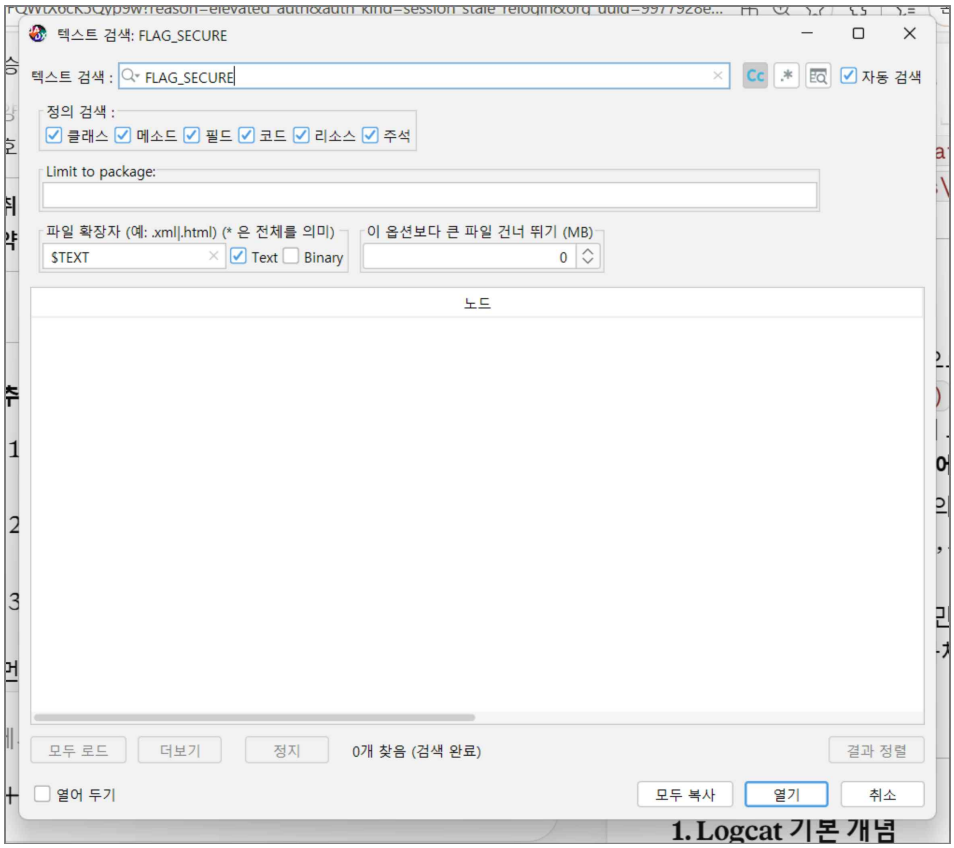
2) release 빌드에서 HttpLoggingInterceptor 레벨을 Level.NONE으로 설정하거나 BuildConfig.DEBUG 조건부로만 인터셉터를 추가할 것.

3) 로깅이 필요하면 비밀번호·토큰·서명 URL 등 민감 필드를 출력 전 마스킹하는 커스텀 인터셉터를 적용할 것.

4.10. 화면 캡처 방지 및 중요정보 마스킹

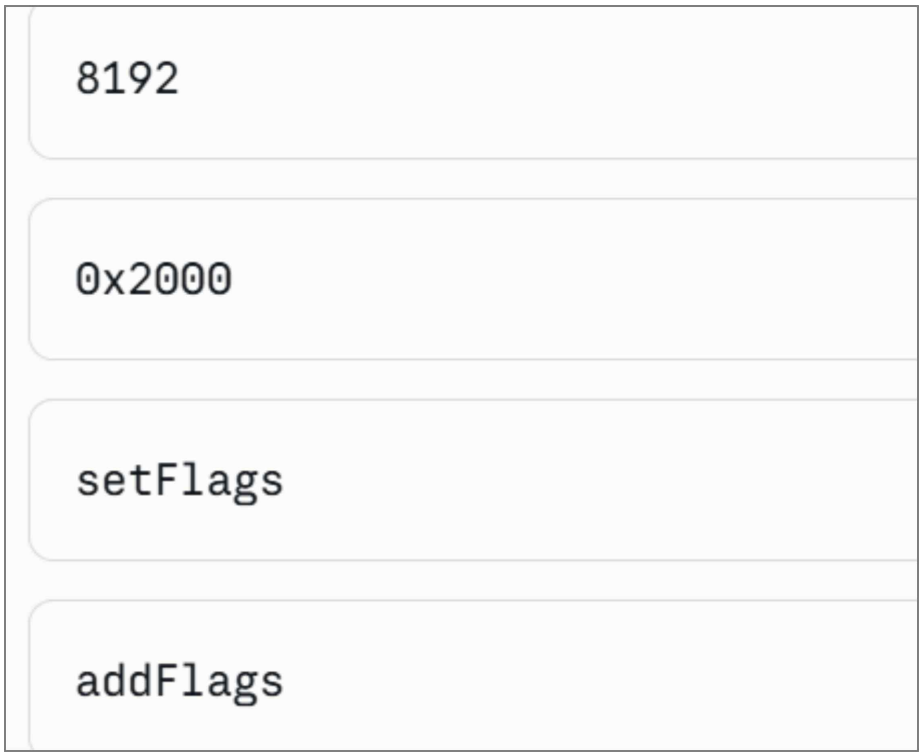
[표 4-] MOB-011 상세 진단 결과

진단 ID	MOB-011	중요도	상
진단 항목	중요정보 표시 화면에서 화면 캡처가 차단되는지 점검		
진단 목적	민감 화면이 캡처되면 화면 내 개인정보가 이미지로 유출됨. 따라서 캡처 방지 적용 여부를 점검함.		
진단 기준	양호	중요 화면에 FLAG_SECURE가 적용되어 캡처 시 정상 이미지가 생성되지 않는 경우	
	취약	중요 화면에서 화면 캡처가 정상적으로 수행되는 경우	
진단 결과	양호		
진단 요약	MyPageActivity에 FLAG_SECURE(8192)가 적용되어 있고 실제 캡처 시 손상 파일이 생성되어 캡처 방지가 정상 동작하므로 양호로 판정한다.		
진단 내용			
1) JADX 정적 분석에서 MyPageActivity.onCreate()에 getWindow().setFlags(8192, 8192)(8192=FLAG_SECURE)가 적용됨을 확인함.			



[그림 4-] FLAG_SECURE 검색 시도

2) 키워드 검색이 실패해 상수값 8192로도 교차 검색하여 FLAG_SECURE 적용을 재확인하고, adb shell screencap 시도 시 정상 이미지가 아닌 손상 파일이 생성되어 캡처가 차단됨을 확인함.



[그림 4- 35] FLAG_SECURE 상수(8192) 교차 확인

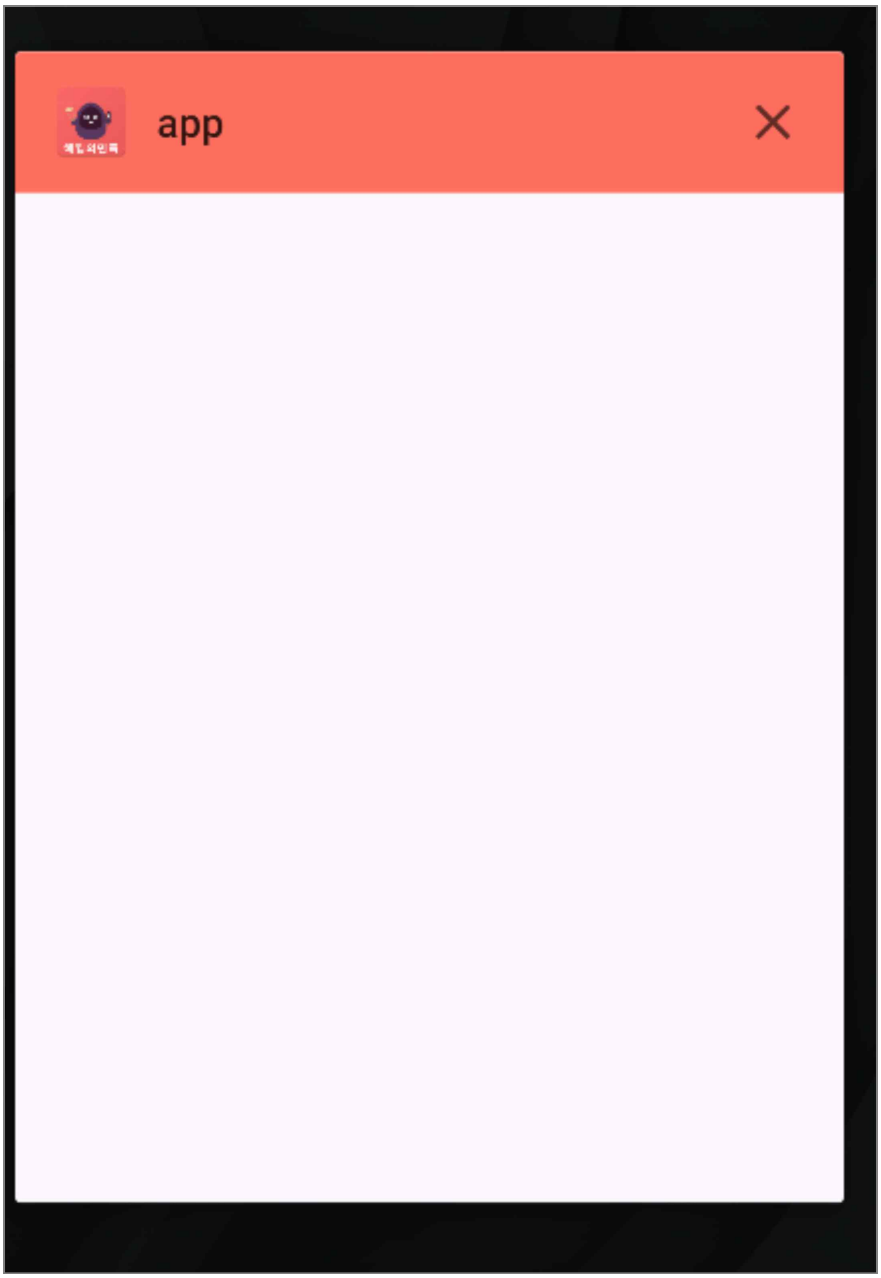
조치 권고

해당 사항 없음.

4.11. 백그라운드 화면 전환 시 정보 보호

[표 4-] MOB-012 상세 진단 결과

진단 ID	MOB-012	중요도	상
진단 항목	앱이 백그라운드로 전환될 때 멀티태스킹 미리보기에 중요정보가 노출되는지 점검		
진단 목적	멀티태스킹 썸네일에 민감 화면이 남으면 정보가 노출됨. 따라서 백그라운드 전환 시 정보 보호 여부를 점검함.		
진단 기준	양호	중요 화면에서 백그라운드 전환 시 썸네일이 가려져(빈 화면) 정보가 노출되지 않는 경우	
	취약	멀티태스킹 미리보기에 중요정보가 그대로 표시되는 경우	
진단 결과	양호		
진단 요약	MyPageActivity 표시 상태에서 멀티태스킹 전환 시 FLAG_SECURE가 썸네일 생성도 차단하여 빈 화면으로 표시되므로 양호로 판정한다.		
진단 내용			
1) MyPageActivity 표시 상태에서 최근 실행 앱(멀티태스킹) 목록으로 전환한 결과, FLAG_SECURE가 썸네일 생성을 차단하여 미리보기가 빈 화면으로 표시되고 중요정보가 노출되지 않음을 확인함.			



[그림 4-] 멀티채팅 스킹 미리보기 썸네일 차단 확인

조치 권고

마이페이지는 보호되나 적용 범위 확대를 권고함.

1) 로그인 등 다른 민감 화면에서도 백그라운드 전환 시 노출이 없도록 FLAG_SECURE 적용 범위를 확대할 것.

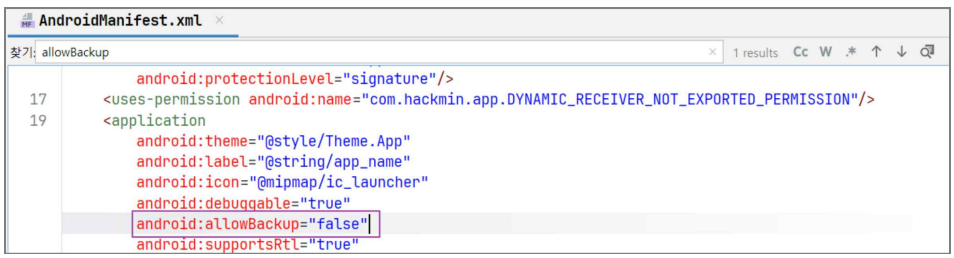
4.12. 앱 백업(AllowBackup) 설정 보안

[표 4-] MOB-013 상세 진단 결과

진단 ID	MOB-013	중요도	상
진단 항목	allowBackup 설정으로 앱 데이터가 외부로 백업·추출 가능한지 점검		
진단 목적	allowBackup이 활성화되면 루팅 없이 adb backup으로 앱 데이터를 외부 추출할 수 있음. 따라서 백업 허용 설정을 점검함.		
진단 기준	양호	android:allowBackup="false"로 설정되어 ADB 백업을 통한 데이터 추출이 차단되는 경우	
	취약	allowBackup="true"이고 민감 파일 제외 규칙이 없어 adb backup으로 앱 데이터가 외부 추출되는 경우	
진단 결과	양호		
진단 요약	AndroidManifest.xml에 android:allowBackup="false"가 설정되어 ADB 백업을 통한 앱 데이터 외부 추출이 차단되므로 양호로 판정한다.		

진단 내용

1) AndroidManifest.xml의 <application> 요소에 android:allowBackup="false"가 설정되어 있어 ADB 백업을 통한 앱 데이터 외부 추출이 차단됨을 확인함.



[그림 4-] AndroidManifest allowBackup=false 설정 확인

조치 권고

해당 사항 없음.

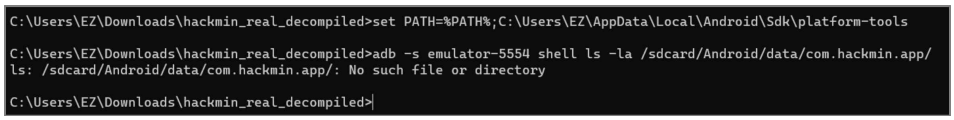
4.13. 외부 저장소(External Storage) 접근 통제

[표 4-] MOB-014 상세 진단 결과

진단 ID	MOB-014	중요도	상
진단 항목	중요 데이터가 외부 저장소에 적재되어 타 앱·사용자에게 노출되는지 점검		
진단 목적	외부 저장소에 민감 데이터를 저장하면 타 앱이 접근할 수 있음. 따라서 외부 저장소 접근·적재 여부를 점검함.		
진단 기준	양호	외부 저장소 권한을 선언하지 않고 민감 데이터를 외부에 적재하지 않는 경우	
	취약	외부 저장소에 민감 데이터를 평문 적재하거나 광범위한 저장소 권한을 요구하는 경우	
진단 결과	양호		
진단 요약	READ/WRITE_EXTERNAL_STORAGE 권한을 선언하지 않고 외부 저장소 경로에 폴더조차 생성하지 않아 민감 데이터 외부 적재 가능성이 없으므로 양호로 판정한다.		

진단 내용

1) /sdcard/Android/data/com.hackmin.app/ 경로 확인 시 No such file or directory로 폴더조차 생성되지 않음을 확인함.



[그림 4-] 외부저장소 권한 미선언 및 경로 부재 확인

조치 권고

해당 사항 없음.

4.14. 통신 구간 암호화 및 취약 프로토콜 제한

[표 4-] MOB-015 상세 진단 결과

진단 ID	MOB-015	중요도	상
진단 항목	통신 구간이 HTTPS로 암호화되고 평문(HTTP) 통신이 제한되는지 점검		
진단 목적	평문 통신은 스니핑·변조에 노출됨. 따라서 통신 암호화 및 취약 프로토콜 제한 여부를 점검함.		
진단 기준	양호	실 통신이 HTTPS로 이루어지고 취약 프로토콜이 제한되는 경우	
	취약	실 통신이 평문(HTTP)으로 이루어지거나 취약 프로토콜을 허용하는 경우	
진단 결과	양호		
진단 요약	매니페스트에 usesCleartextTraffic="true"가 설정되어 있으나 최종 APK의 실제 통신은 HTTPS(https://hackmin.com)로 확인되어 통신 구간이 암호화되므로 양호로 판정한다.		

진단 내용

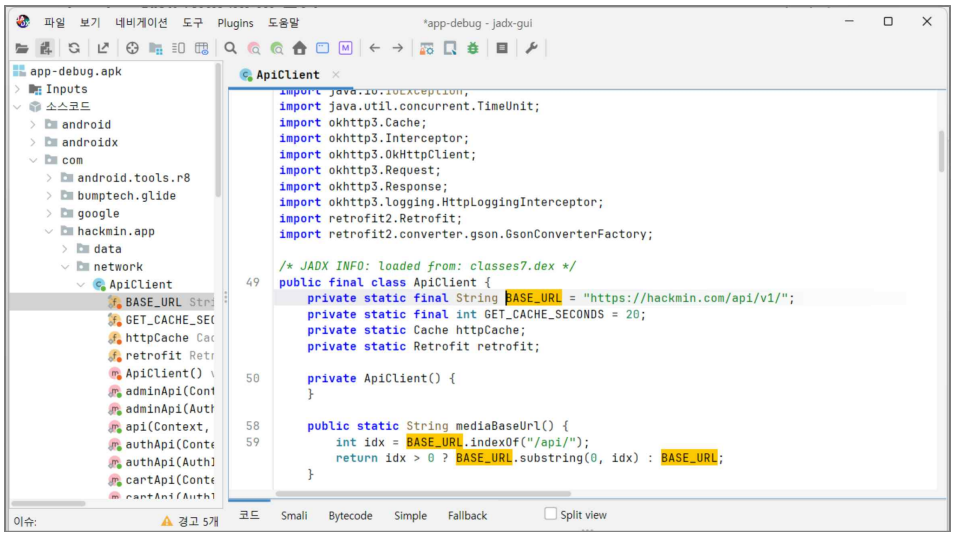
1) AndroidManifest.xml에 android:usesCleartextTraffic="true"가 설정되어 앱 전체가 HTTP 평문 통신을 허용하도록 되어 있음을 확인함(하드닝 관점의 개선 필요 사항).

```
C:\Users\EZ>cd C:\Users\EZ\Downloads

C:\Users\EZ\Downloads>findstr /i "usesCleartextTraffic networkSecurityConfig" hackmin_real_decompiled\AndroidManifest.xml
    <application android:allowBackup="true" android:appComponentFactory="androidx.core.app.CoreComponentFactory" android:dataExtractionRules="@xml/data_extraction_rules" android:debuggable="true" android:extractNativeLibs="true" android:fullBackupContent="@xml/backup_rules" android:icon="@mipmap/ic_launcher" android:label="@string/app_name" android:roundIcon="@mipmap/ic_launcher_round" android:supportsRtl="true" android:theme="@style/Theme.App" android:usesCleartextTraffic="true">
```

[그림 4-] 매니페스트 usesCleartextTraffic 설정 확인

2) 최종 APK의 ApiClient 및 실제 통신 엔드포인트를 확인한 결과 HTTPS로 통신이 이루어져 실 통신 구간은 암호화됨을 확인함.



[그림 4-] 실제4통신 엔드포인트 HTTPS 확인

조치 권고

해당 사항 없음

4.15. 서버 인증서 검증 및 고정(Pinning)

[표 4-] MOB-016 상세 진단 결과

진단 ID	MOB-016	중요도	상
진단 항목	서버 인증서 검증 및 인증서 고정(Pinning)이 적용되어 MITM을 방지하는지 점검		
진단 목적	핀닝이 없으면 시스템 CA 우회·MITM으로 트래픽이 복호화될 수 있음. 따라서 인증서 검증·고정 적용 여부를 점검함.		
진단 기준	양호	CertificatePinner 또는 network_security_config의 pin-set 등으로 인증서 고정이 적용된 경우	
	취약	핀닝이 부재하여 시스템 CA/프록시로 트래픽 복호화가 가능한 경우	
진단 결과	양호		
진단 요약	정적 분석에서 ApiClient에 CertificatePinner가 적용되어 인증서 고정이 확인되므로 양호로 판정한다.		
진단 내용 (증적)			
1) 정적 분석 결과 ApiClient에 CertificatePinner를 사용한 SSL 인증서 고정(Pinning)이 적용되어 있음을 확인함.			
조치 권고			
핀닝이 적용되어 있으나 운영 관점에서 다음을 권고함. 1) 핀 로테이션·백업 핀 관리 정책을 유지하고, 핀닝 로직을 네이티브(C/C++)로 구현하여 Frida 우회 난이도를 높일 것. 2) 클라이언트 핀닝은 우회 가능성이 있으므로 서버 측 mTLS 등을 병행 검토할 것.			

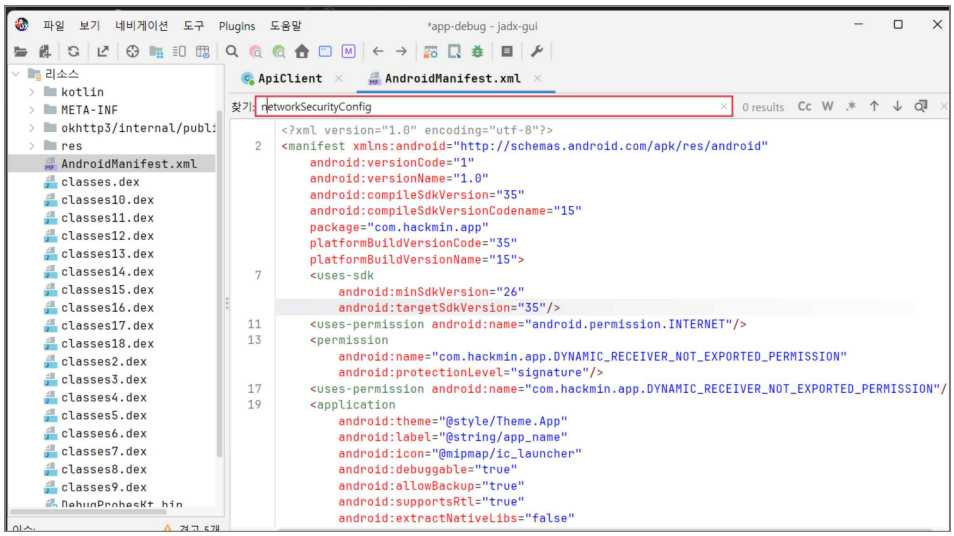
4.16. 앱 컴포넌트(IPC) 강제 실행 및 노출 방지

[표 4-] MOB-017 상세 진단 결과

진단 ID	MOB-017	중요도	상
진단 항목	Activity/Service/Receiver/Provider의 외부 노출(exported) 설정이 최소화되어 있는지 점검		
진단 목적	불필요하게 exported된 컴포넌트는 외부 앱이 직접 호출·조작할 수 있음. 따라서 컴포넌트 노출 설정을 점검함.		
진단 기준	양호	런처 진입점 등 필수 컴포넌트만 exported이고 비즈니스 로직 컴포넌트는 exported=false인 경우	
	취약	비즈니스 로직 컴포넌트가 불필요하게 외부 노출된 경우	
진단 결과	양호		
진단 요약	exported=true인 컴포넌트가 런처 진입점(LoginActivity)과 signature 권한으로 보호된 표준 Receiver뿐이며 비즈니스 로직 컴포넌트는 모두 exported=false로 확인되어 양호로 판정한다.		

진단 내용

1) AndroidManifest.xml의 컴포넌트 exported 설정을 전수 확인한 결과, exported=true는 LoginActivity(MAIN/LAUNCHER 진입점)와 ProfileInstallReceiver(AndroidX 표준, DUMP signature 권한 보호)뿐이며 주문·결제·채팅·마이페이지 등 비즈니스 로직 컴포넌트는 모두 exported=false로 외부 직접 호출이 차단됨을 확인함.



[그림 4-] 컴포넌트 exported 설정 전수 확인

조치 권고

해당 사항 없음.

1) 신규 컴포넌트도 기본값 exported=false를 유지하고, 외부 노출이 필요한 경우에만 최소 명시 허용할 것.

4.17. 모바일 딥링크(DeepLink) 도용 제한

[표 4-] MOB-018 상세 진단 결과

진단 ID	MOB-018	중요도	상
진단 항목	딥링크를 통한 앱 내 화면 강제 호출·파라미터 주입이 가능한지 점검		
진단 목적	취약한 딥링크는 외부 URI로 앱 내 화면을 강제 호출하거나 파라미터를 주입할 수 있음. 따라서 딥링크 설정을 점검함.		
진단 기준	양호	커스텀 스키마·App Links·BROWSABLE 딥링크가 없거나 안전하게 통제되는 경우	
	취약	검증 없는 딥링크로 내부 화면 강제 호출·파라미터 주입이 가능한 경우	
진단 결과	양호		
진단 요약	매니페스트 전체 intent-filter 확인 결과 커스텀 스키마·App Links·BROWSABLE 딥링크가 존재하지 않아 외부 URI를 통한 공격이 성립하지 않으므로 양호로 판정한다.		
진단 내용			
1) AndroidManifest.xml의 전체 intent-filter를 확인한 결과 커스텀 스키마(android:scheme)·App Links·BROWSABLE 딥링크 설정이 존재하지 않으며, 등록된 intent-filter는 LoginActivity의 MAIN/LAUNCHER와 AndroidX 표준 액션뿐으로 외부 URI를 통한 화면 강제 호출·파라미터 주입 경로가 부재함을 확인함.			
조치 권고			
해당 사항 없음.			

4.18. WebView 보안 및 JS 인터페이스 통제

[표 4-] MOB-019 상세 진단 결과

진단 ID	MOB-019	중요도	상
진단 항목	WebView의 JavaScript 활성화·인터페이스 노출 및 URL 로딩 통제가 안전한지 점검		
진단 목적	통제되지 않은 WebView는 스크립트 조작·브릿지 오용에 노출됨. 따라서 WebView 보안 설정을 점검함.		
진단 기준	양호	URL 화이트리스트·MixedContent 차단·브릿지 최소 노출이 적용된 경우	
	취약	URL 로딩 통제가 없거나 MixedContent를 허용하여 스크립트 조작·브릿지 오용이 가능한 경우	
진단 결과	취약		
진단 요약	주소 검색 WebView에서 JS가 활성화되고 브릿지가 노출된 상태로 shouldOverrideUrlLoading 미오버라이드 및 MixedContent 완화 설정이 확인되어 통신 변조 시 스크립트 조작 가능성이 있으므로 취약으로 판정한다.		

진단 내용

1) WebView는 우편번호 주소 검색(PostcodeSearch) 1곳에서 사용되며, setJavaScriptEnabled(true)로 JS가 활성화되고 addJavascriptInterface로 네이티브 브릿지가 노출됨을 확인함. shouldOverrideUrlLoading()이 미오버라이드되어 URL 화이트리스트 통제가 부재하고 MixedContent 완화 설정이 확인되어, 통신 변조(MITM)를 통한 스크립트 조작·브릿지 오용 가능성이 있음.

```
public static void L(Activity activity, i2 i2Var) {
    if (w3.a()) {
        WebView webView = new WebView(activity);
        WebSettings settings = webView.getSettings();
        settings.setJavaScriptEnabled(true);
        settings.setDomStorageEnabled(true);
        settings.setMixedContentMode(2);
        webView.setWebViewClient(new WebViewClient());
        Dialog dialog = new Dialog(activity);
        dialog.setContentView(webView);
        if (dialog.getWindow() != null) {
            dialog.getWindow().setLayout(-1, (int) (((double) activity.getResources().getD
        }
        webView.addJavascriptInterface(new s10(activity, i2Var, dialog), "AndroidBridge");
        webView.loadDataWithBaseURL("https://t1.daumcdn.net", "<!DOCTYPE html><html><head>
        dialog.show();
    }
```

[그림 4- 42] WebView JS·브릿지 설정 및 URL 통제 부재 확인

조치 권고

완화 요소(브릿지가 주소 콜백 1개로 제한, minSdk 26)가 있으나 다음을 권고함.

1) shouldOverrideUrlLoading()에서 신뢰 도메인(t1.daumcdn.net, .daum.net) 화이트리스트로 URL 이동을 제한할 것.

2) setMixedContentMode를 MIXED_CONTENT_NEVER_ALLOW로 변경할 것.

3) 브릿지 노출 함수는 최소(주소 콜백)로 유지하고 파일·설정·개인정보 접근 기능 추가를 금지할 것.

4.19. 안전하지 않은 동적 코드 로드 방지

[표 4-] MOB-020 상세 진단 결과

진단 ID	MOB-020	중요도	상
진단 항목	외부에서 실행 코드를 내려받아 런타임에 로드하는 위험한 동적 로딩이 있는지 점검		
진단 목적	검증되지 않은 동적 코드 로딩은 임의 코드 실행으로 이어질 수 있음. 따라서 동적 코드 로딩 사용 여부를 점검함.		
진단 기준	양호	DexClassLoader 등 외부 코드 동적 로딩이 없고 네이티브 로드도 보호 경로에서만 이루어지는 경우	
	취약	외부 서버·저장소에서 실행 코드를 내려받아 런타임 로드하는 경로가 존재하는 경우	
진단 결과	양호		
진단 요약	동적 코드 로딩 API(DexClassLoader 등) 사용이 없고 System.loadLibrary만 존재하여 안드로이드 보호 경로에서만 로드되므로 양호로 판정한다.		

진단 내용

1) JADX로 동적 코드 로딩 API(DexClassLoader, PathClassLoader, URLClassLoader, System.load)를 전수 검색한 결과 앱 자체 코드 내 사용이 없으며, System.loadLibrary(자체 네이티브 로드)만 존재하고 안드로이드 관리 보호 경로에서만 로드되어 임의 조작이 불가함을 확인함.

조치 권고

해당 사항 없음.

1) 향후 플러그인·핫픽스로 동적 로딩 도입 시 코드 서명 검증·신뢰 소스(자체 서버·HTTPS)로 제한할 것.

4.20. 과도한 권한(Permission) 요청 통제

[표 4-] MOB-021 상세 진단 결과

진단 ID	MOB-021	중요도	상
진단 항목	앱이 기능에 불필요한 과도한 권한을 요구하는지 점검		
진단 목적	과도한 권한은 오남용 시 개인정보 침해로 이어짐. 따라서 최소 권한 원칙 준수 여부를 점검함.		
진단 기준	양호	기능에 필요한 최소 권한만 선언하고 불필요한 Dangerous 권한이 없는 경우	
	취약	기능과 무관한 Dangerous 권한(카메라·위치·연락처 등)을 과도하게 선언한 경우	
진단 결과	양호		
진단 요약	선언 권한이 INTERNET(normal)과 AndroidX 자동 생성 signature 권한 2개뿐이며 Dangerous 권한이 없어 최소 권한 원칙을 준수하므로 양호로 판정한다.		

진단 내용

1) AndroidManifest.xml의 uses-permission을 전수 확인한 결과 선언 권한은 android.permission.INTERNET(normal)과 AndroidX 자동 생성 DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION(signature) 2개뿐이며, 카메라·위치·연락처·외부저장소 등 Dangerous 권한이 미선언되어 최소 권한 원칙을 준수함을 확인함.

```
C:\Users\EZ\Downloads>cd C:\Users\EZ\Downloads
C:\Users\EZ\Downloads>findstr /i "networkSecurityConfig" hackmin_real_decompiled\AndroidManifest.xml
```

[그림 4-] 매니페스트 uses-permission 전수 확인

조치 권고

해당 사항 없음.

1) 신규 기능 추가 시에도 최소 권한만 선언하고 런타임 권한 모델로 사용 시점에 동적 요청할 것.

4.21. 안전한 자격 증명(Credential) 및 API 키 관리

[표 4-] MOB-022 상세 진단 결과

진단 ID	MOB-022	중요도	상
진단 항목	API 키·시크릿·자격증명이 안전하게 관리되고 하드코딩·평문 저장되지 않는지 점검		
진단 목적	하드코딩된 키·시크릿은 디컴파일로 탈취됨. 따라서 자격증명·API 키 관리 상태를 점검함.		
진단 기준	양호	키·시크릿이 하드코딩되지 않고, 카드정보 등이 로컬에 평문 저장되지 않는 경우	
	취약	키·시크릿·자격증명이 소스에 하드코딩되거나 민감정보가 평문 저장되는 경우	
진단 결과	양호		
진단 요약	key·secret·apikey·password 등을 전수 검색한 결과 하드코딩된 자격증명이 발견되지 않고 카드정보도 로컬 미저장이 확인되어 양호로 판정한다.		
진단 내용			
1) JADX로 key·secret·apikey/api_key·password·Base64를 소스·리소스 전 범위에서 검색한 결과 하드코딩된 키·시크릿·자격증명 값이 발견되지 않았으며, 검색 결과는 라이브러리 리소스 ID·DTO 필드명·API 경로로 분류됨(카드정보는 PaymentCardDto에서 마스킹된 형태로만 다뤄짐).			

PaymentCardDto

```
package com.hackmin.app.data.model.user;

import com.google.gson.annotations.SerializedName;
import com.hackmin.app.ui.order.EasyPayActivity;

/* JADX INFO: loaded from: classes6.dex */
public class PaymentCardDto {

    @SerializedName("card_masked")
    private String cardMasked;

    @SerializedName("id")
    private long id;

    @SerializedName(EasyPayActivity.EXTRA_PROVIDER)
    private String provider;

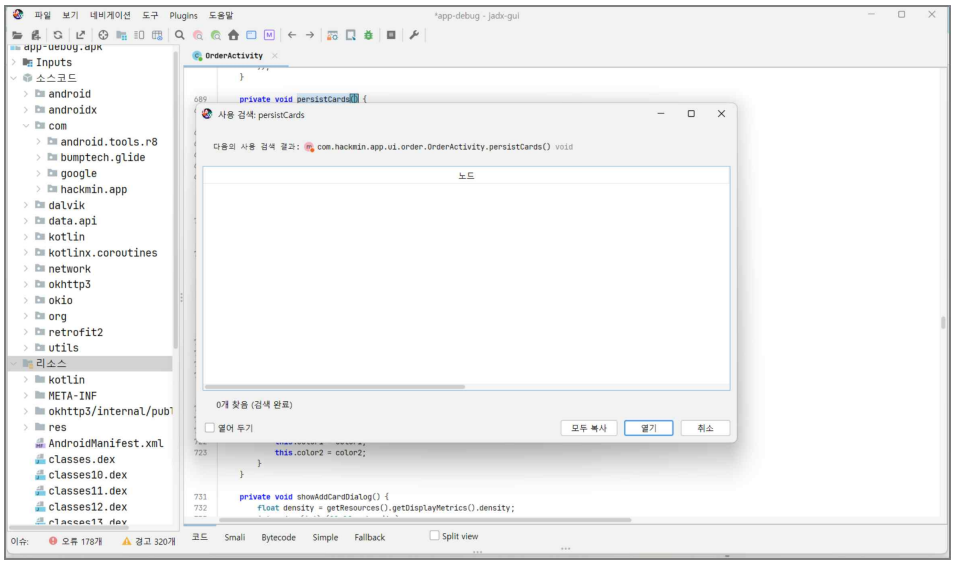
    public long getId() {
        return this.id;
    }

    public String getProvider() {
        return this.provider;
    }

    public String getCardMasked() {
        return this.cardMasked;
    }
}
```

[그림 4-] 자격증명 검색 결과 분류(하드코딩 부재)

2) 카드정보의 로컬 영속 저장 여부를 확인한 결과 persistCards 사용처가 0건으로, 카드번호가 로컬에 저장되지 않음을 확인함.



[그림 4-] persistCards 사용처 0건(카드정보 로컬 미저장)

조치 권고

본 항목 자체의 특이사항은 없으나 다음을 권고함.

1) 향후 결제·지도·FCM 등 외부 연동 시 API 키를 소스/리소스에 평문 하드코딩하지 말고 Android Keystore 암호화 저장·서버 프록시·NDK 난독화 저장을 적용할 것.

5. 별첨

5.1. 용어 정의

[표 5-] 용어 정의

용어	설명
루팅(Rooting)	Android 단말에서 관리자(root) 권한을 획득하는 행위
후킹(Hooking)	실행 중인 프로그램의 함수 호출을 가로채 동작을 변경하는 기법
Frida	런타임에 프로세스를 계측·후킹하는 동적 분석 도구
FLAG_SECURE	화면 캡처·미리보기 생성을 차단하는 Android 윈도우 플래그
프리사인드 URL	일정 시간 동안 인증 없이 객체 접근을 허용하는 서명된 URL